

From ATOP to ZCube: Automated Topology Optimization Pipeline and A Highly Cost-Effective Network Topology for Large Model Training

Zihan Yan¹, Dan Li¹, Li Chen², Dian Xiong³, Kaihui Gao², Yiwei Zhang¹, Rui Yan¹,
Menglei Zhang⁴, Bochun Zhang⁴, Zhuo Jiang⁴, Jianxi Ye⁴, Haibin Lin⁴
¹Tsinghua University ²Zhongguancun Laboratory ³Harnets.AI ⁴ByteDance

Abstract

The development of large language models (LLMs) poses new challenges in data center network topology design. To assist in exploring topology design, we propose ATOP, an Automated Topology Optimization Pipeline, which models network topology as a set of hyperparameters, enabling the discovery of potential topologies. With various optimization algorithms and customizable optimization objectives, ATOP achieves automated topology optimization on a scale of tens of thousands of GPUs. We apply ATOP on network topologies for 256, 1024, 4096, and 16384 GPUs, optimizing performance under LLMs training traffic patterns, collective communication performance, fault tolerance, and network cost. We also evaluate ATOP in different scenarios: building, optimizing, and expanding a data center. From ATOP's results, we discover a new topology – ZCube, which reaches the highest cost-effectiveness across various GPU scales. Simulation results show that ZCube, compared to the previous state-of-the-art topologies, including Rail-optimized Fat-tree (ROFT), Rail-only, and HPN, improves end-to-end LLM training speed by 3% to 7% and reduces network hardware costs by 26% to 46%. We also construct ZCube on a real-world testbed. Results show that ZCube reduces hardware costs by 25% compared to Rail-Optimized Topology while maintaining the same all-reduce and all-to-all performance.

CCS Concepts

• **Networks** → **Data center networks**; **Topology analysis and generation**.

Keywords

Data center networks, Network topology, AI infrastructure

ACM Reference Format:

Zihan Yan, Dan Li, Li Chen, Dian Xiong, Kaihui Gao, Yiwei Zhang, Rui Yan, Menglei Zhang, Bochun Zhang, Zhuo Jiang, Jianxi Ye, Haibin Lin. 2025. From ATOP to ZCube: Automated Topology Optimization Pipeline and A Highly Cost-Effective Network Topology for Large Model Training. In *ACM SIGCOMM 2025 Conference (SIGCOMM '25)*, September 8–11, 2025, Coimbra, Portugal. ACM, New York, NY, USA, 21 pages. <https://doi.org/10.1145/3718958.3750503>



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

SIGCOMM '25, Coimbra, Portugal

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1524-2/25/09

<https://doi.org/10.1145/3718958.3750503>

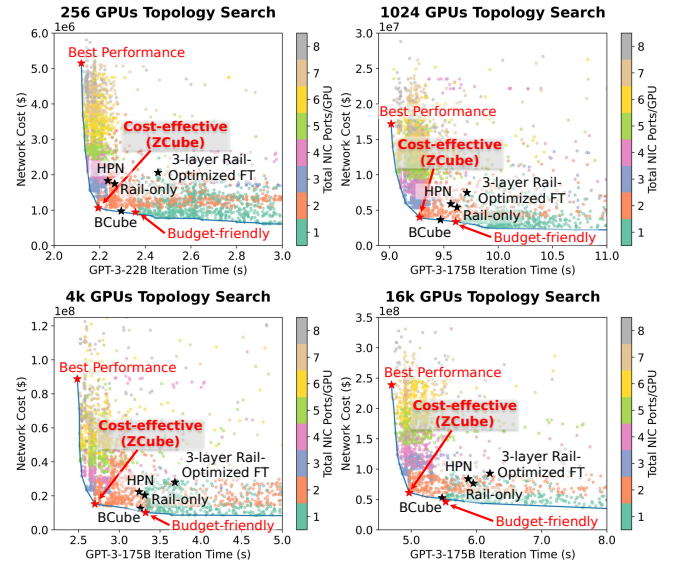


Figure 1: ATOP search results on different GPU scales, each point representing a topology. For each scale, we label the three notable points in each plot: Best performance, Most Cost-effective, and Budget-friendly.

1 Introduction

With the emergence of generative AI, the exponential growth of model sizes has led to the construction of increasingly large and expensive training clusters. For example, ByteDance has built a cluster of over 10k GPUs [29]. Meta built a cluster of 24k GPUs for Llama-3 training [23]. X.AI has also invested at least \$3 billion [18] in AI infrastructure.

Designing inter-server topologies for large clusters requires optimizing multiple objectives, and the most important ones include performance, fault tolerance, and cost. There are complex trade-offs among these objectives, and it is virtually impossible to achieve optimality in all dimensions. Therefore, it is practical to seek cost-effective topologies—those that offer the best performance within a given budget or the lowest cost for a given performance level, *i.e.*, Pareto-optimal [9] topologies.

Current topology design processes rely on selecting topologies from the HPC and DCN literature [23, 29]. However, we find that well-known topologies fall short of Pareto-optimality. We conduct extensive experiments showing that Rail-optimized Fat-tree (ROFT) [39], Rail-only [51], and HPN 7.0 [44] are not Pareto-optimal. Fig. 1 showcases the Pareto frontier of the trade-off between performance (GPT-3 model training iteration time (x-axis)) and total

network hardware cost (y-axis). Each point represents a topology, and we defer the detailed generation method of each point to §3. The results reveal that many alternative designs outperform these state-of-the-art topologies in both cost and performance.

Topology design is also not a “one-time” deal. There are at least three operational scenarios that necessitate design processes: (1) Initial construction, where operators architect new data centers to meet specific requirements, such as new model architectures (e.g., Mixture-of-Experts models [33]) and new networking hardware (e.g., next generation switching chips [22]); (2) Incremental adjustments, e.g., re-cabling to mitigate bottlenecks or reduce operational costs; and (3) Expansion, e.g., scaling from 1k to 4k GPUs while preserving fault tolerance. Automating the design process is thus a practical and worthwhile pursuit.

Therefore, cluster topology design is a practical and challenging task involving multiple objectives: end-to-end performance of target workloads, fault tolerance, cost constraints, hardware availability, *etc.* There are currently two main design approaches: manual and automated. Manual design by human experts is the most common approach, but it suffers from three issues.

- The topology design space for large-scale clusters is vast, making it difficult for manual design to efficiently explore the topology space.
- Human experts are biased towards aesthetically pleasing topologies and may overlook non-intuitive topologies with better properties.
- Manual design also struggles to account for multiple objectives simultaneously.

Thus, both industry and academia are investigating automated topology design methods. The naïve method is to model topologies using adjacency matrices, but this introduces an enormous search space, which is impractical to search. The seminal work, Condor [48], addresses this issue by introducing a domain-specific language (DSL) of topology constraints and models the topology design as a constraint satisfiability problem. However, it still relies on human experts to express their intuition using the DSL. Although the DSL makes it easy for experts to describe hardware constraints (e.g., per switch port count) and failure tolerance properties (e.g., maintain full connectivity in single-switch failures), it is very difficult and unintuitive to express application-level, end-to-end performance optimization goals (e.g., minimizing LLM training iteration time) using a constraint language.

Thus, we ask the question: *can we combine human intuitions and automated exploration in cluster topology design with multiple objectives?* To this end, we propose ATOP, an Automated Topology Optimization Pipeline. Specifically, ATOP addresses the limitations of existing topology design methods from three key aspects.

- **Intuition-driven Modeling:** We study all the popular DCN and HPC topologies, and distill the expert intuitions behind each design. We then formalize the intuitions into 11 types of searchable hyperparameters, which construct a reasonably large topology space. ATOP’s modeling approach covers most known existing topologies, such as CLOS [13, 23, 29], Fat-Tree [2], ROFT [39], Rail-only [51], HPN [44], BCube [25], DCell [26], HyperX [1], Torus [30], Dragonfly [32], *etc.* The topologies generated by ATOP are not limited to variants or

combinations of these existing designs but are capable of producing new asymmetric topologies, providing a high degree of freedom in the search space while reasonably constraining its scope.

- **Multi-Objective Optimization via Guided Simulation:** ATOP combines the NSGA-II evolutionary algorithm [15] with a high-performance, high-fidelity simulator to efficiently explore topology designs. The optimizer iteratively generates candidate topologies using non-dominated sorting and crowding distance metrics, while the simulator uses realistic workloads to evaluate each candidate against user-defined objectives—including throughput, fault tolerance, cost, and network diameter.
- **Efficiency and Scalability:** ATOP improves efficiency and scalability in three aspects: Firstly, our topology modeling approach reasonably reduces the search space. Secondly, we improve the scalability of Astra-Sim [54] with a flow-level simulator which emphasizes fidelity. This simulator can efficiently evaluate the performance of large-scale topologies with real LLM training workloads. Finally, we design a 2-stage evaluation method to significantly reduce the number of evaluations: the first stage employs representative fragments of model training workloads, producing a Pareto-optimal set of topologies. The second stage runs a complete LLM training workload for each of the topologies to obtain the final results. With these techniques, ATOP is able to discover new superior topologies within 3 days on a single CPU server for a cluster with 16384 GPUs.

We evaluate ATOP in the above three design scenarios (§4). For example, in the design of initial construction, we use ATOP to optimize topology for varying cluster sizes (from 256 to 16384).

From the results, we are surprised to discover that, given practical performance objectives (LLM training iteration time, real network cost models, and physical constraints of current hardware), the most cost-effective topologies among all the topologies we have searched in the three scenarios are similar, and, to the best of our knowledge, have not been disclosed. We then formalize this new topology, and name it *ZCube*. We verify the performance of ZCube through packet-level simulation, theoretical analysis, and building a real testbed with 16 GPUs and 32×200GbE ports. We run the ncc1-tests [40], and find that ZCube reduces hardware costs by 25% compared to non-block ROFT while maintaining the same all-reduce and all-to-all performance.

The main contributions of our work are as follows.

- We summarize the expert insight of most existing DCN and HPC topologies, and propose a new abstract model with 11 types of searchable hyperparameters, thereby obtaining a reasonably sized search space that includes all popular topologies.
- We propose ATOP, a novel topology design system. It achieves multi-objective optimization with a high-performance, high-fidelity simulator guided by an evolutionary algorithm. Users can freely define objectives and constraints for ATOP to adapt to different scenarios.
- We develop a 2-stage evaluation method that can reduce the number of full-scale model training simulations necessary for ATOP’s topology optimization by 95%. ATOP is able to discover new superior topologies within 3 days on a single CPU server for a cluster with 16384 GPUs.

- Using ATOP, we discover ZCube, a new topology that exhibits low diameter, strong LLM training performance, robust fault tolerance, and cost-effectiveness across different scales of GPUs. Simulation results show that ZCube, compared to the previous state-of-the-art topologies, including ROFT, Rail-only, and dual-port designs for ROFT (HPN), improves end-to-end LLM training speed by 3% to 7% and reduces network hardware costs by 26% to 46%. We also construct ZCube on a real-world testbed. Results show that ZCube reduces hardware costs by 25% compared to Rail-Optimized Topology while maintaining the same all-reduce and all-to-all performance.

Limitations. This paper proposes ATOP for automated topology optimization, and a new cost-effective topology, ZCube. Our limitations are as follows and we seek to improve them in future work. ATOP’s topology modeling does not generate arbitrary topologies, as it leverages prior knowledge in topology design to achieve a trade-off between the size of the search space and search efficiency. Despite this, ATOP has discovered numerous novel topologies that outperform existing ones in both performance and cost-effectiveness. Secondly, the optimization results are also limited by the optimization algorithm. While we achieve good results using the NSGA-II algorithm in this paper, larger sampling sizes or more powerful optimization algorithms may lead to better results. Thirdly, due to time and resource constraints, we optimize for topologies of up to 16384 GPUs, and ATOP can be used for larger scale in future work. We also cannot test ZCube on a larger real testbed with more servers, and to verify its performance at large scale; we can only rely on packet-level discrete-event simulation.

Ethics: *This work raises no ethical concerns.*

2 Background and Motivation

In this section, we introduce the background of large model training, large-scale AI cluster topologies, design methods, and motivate the design of ATOP.

LLM Training Traffic Patterns. Distributed training techniques parallelize the LLM training workload, each generating distinct communication patterns. Understanding these patterns is crucial for designing efficient cluster topologies. To study these communication patterns, we record traffic traces from large-scale training runs on a production cluster with 4k GPUs. We also use the AICB workload generator in SimAI [3] and conduct interviews with LLM researchers and practitioners. We focus only on the inter-server traffic; thus we exclude tensor parallelism (TP).

- **Data parallelism (DP)** partitions training data across GPUs. Each GPU computes gradients on its local partition, which are then aggregated to update global model parameters, generating all-gather (AG) and reduce-scatter (RS) collective communication [35], with message size from 10 to 100 MB.
- **Pipeline parallelism (PP)** splits the model across devices, with each GPU handling a layer or set of layers. Inputs pass forward through this pipeline, with each GPU performing its computation and passing activations to the next GPU. Gradients are passed backward through the same pipeline, generating point-to-point (P2P) communication [35], with message size 1-10 MB.



Figure 2: GPT-3 training timeline on rank 0 of classical interleaved 1F1B schedule, excluding TP communication as it typically occurs on the intra-server network.

- **Expert parallelism (EP)**, used in mixture-of-expert (MoE) models, generates all-to-all traffic among GPUs [45]. We learn from LLM researchers and training frameworks that EP traffic may co-exist with DP traffic but rarely with PP traffic [45, 49, 53, 55, 57].
- **Co-existence.** By analyzing actual traffic traces (e.g., Fig. 2), we find that, during the actual training, five types of traffic periods exist: DP-only, PP-only, EP-only, mixed DP-PP, and mixed DP-EP.

We find that these traffic share a common characteristic: cross-server communication typically occurs between GPUs with the same index (e.g., from GPU_2 on Server_1 to GPU_2 on Server_64). This remains true for all-to-all communication because PXN in NCCL first forwards traffic to the same-index GPU using NVLink and then to the destination server [42]. This traffic characteristic is called *rail-aligned* [51]. In contrast, *inter-rail* traffic refers to inter-server communication that occurs between GPUs with different indexes. We observe that rail-aligned traffic also traverses spine switches in ROFT because the communicating servers may belong to different pods, for example, PP traffic and collective communication generated by large DP group typically. During one month of monitoring in a production GPU cluster with ROFT topology, we observe that 35% of rail-aligned traffic traverses spine switches.

With the above analysis, we prepare representative workloads for each traffic period to use in the 2-stage evaluation mechanism of ATOP and ensure that all traffic is rail-aligned and there is no inter-rail traffic.

Large-Scale GPUs Interconnection Topology. Training LLMs requires thousands of GPUs (1k ~ 10k) [35, 45], making efficient GPU interconnection critical. Intra-server GPU connections use NVLink or PCIe, while inter-server connections rely on Ethernet, Infiniband (IB), or RDMA over Converged Ethernet (RoCE). As intra-server networks are usually optimized by manufacturers, we focus on inter-server network topologies. Currently, the most commonly used topologies in practice include CLOS and ROFT [23, 29, 39]. ROFT connects GPUs with the same index in different servers via a single ToR switch (e.g., the GPU_0 of all the servers in the same pod are connected). Rail-only constructs a Rail-interconnection for GPUs with the same index on each server, eliminating the inter-rail interconnection. Alibaba’s HPN [44] uses dual-port NICs to build a dual-port designs for ROFT (dual-plane architecture). In Appendix K, we provide diagrams illustrating ROFT, Rail-only, and HPN topologies built with 51.2 Tbps switches for 16k GPU clusters.

Current topologies make different trade-offs between performance, cost, and fault tolerance.

- **Performance** drives topology design toward full bisection bandwidth. ROFT [23, 29, 39], Rail-only, and HPN [44] all achieve this theoretical maximum. However, theoretical performance does

not guarantee practical performance. ROFT suffers from load balancing inefficiencies under EP-driven all-to-all traffic due to ECMP hash collision and hash polarization. Fig. 3(a) shows the maximum number of conflicting flows per bandwidth for 256 GPUs under three scenarios: "Ideal LB", "ECMP + PXN ON", and "ECMP + PXN OFF". NCCL PXN [42] provides performance improvement, while topology design offers greater gains.

- *Cost constraints* push designers toward reduced hardware requirements. BCube reduces hardware costs but cannot provide full bisection bandwidth, hurting MoE model training. ROFT and Rail-only achieve full bisection bandwidth with standard hardware costs. HPN improves fault tolerance but increases network hardware costs by 10% compared to Rail-only due to its dual-plane architecture.
- *Fault tolerance* is another key concern of topology design. ROFT and Rail-only exhibit poor fault tolerance: a single ToR failure affects many GPUs, forcing them to use PXN for traffic forwarding through intra-server interconnects. This creates intra-server bandwidth competition and impacts other parallel strategies. Fig. 3(b) shows that one ToR failure in a 4k GPU cluster degrades LLM training performance by 46.9% in ROFT and 46.2% in Rail-only. HPN addresses this vulnerability through its dual-plane architecture but at increased cost.

The complex trade-off between performance, cost, and fault tolerance motivates us to develop an automated topology design tool that identifies the Pareto-optimal topologies and discovers new topologies that combine high cost-effectiveness with strong fault tolerance.

Existing Topology Design Methods. There are currently two main approaches for topology design: manual and automated. For the manual method, designers often rely on expert knowledge to select a highly symmetric topology. This focus on symmetry can overlook potentially high-performance asymmetric topologies. Additionally, designing a topology that meets multiple performance criteria is challenging, such as when ROFT excels in all-reduce but falls short in cost and fault tolerance.

For the automated method, one naïve approach involves using adjacency matrix-based topology modeling for exhaustive search [14], but this results in an infeasibly large search space. For N GPUs, the space grows as $O(2^{N^2})$, making it impractical to quickly find superior designs. For instance, we implement a program for searching 16k GPU topologies using an adjacency matrix, which takes 10 hours without producing a connected topology, as hundreds of millions of parameters are required to represent the topology. Transforming a ROFT topology into Dragonfly requires modifying hundreds of millions of parameters.

Previous seminal work Condor [48] has partially addressed the problem of automated design. Schlinder *et al.* developed a topology description language and introduced constraint solving to this area. However, since describing topology features that benefit application performance still relies on human intuition, this method cannot directly reason about end-to-end application performance.

Thus, we conclude that existing approaches are reaching their limits as the demands of modern workloads grow in complexity and scale. Naïve automated approaches lack the domain insights to effectively constrain the search, while manual design relies on

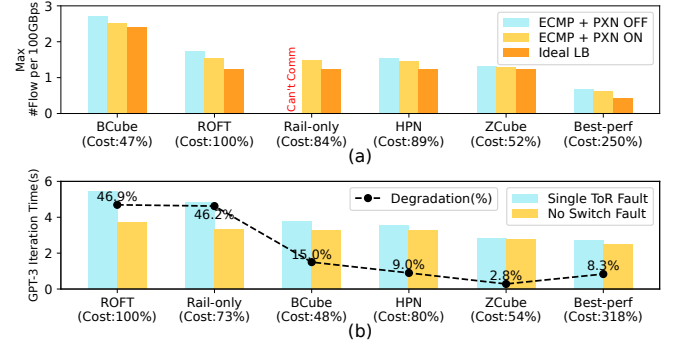


Figure 3: (a) The max number of flow per 100 Gbps under all-to-all traffic in a 256-GPUs topology. (b) The performance degradation of GPT-3 training after a Single ToR Fault in a 4k-GPUs topology. ZCube and Best-Perf are new topologies generated by ATOP.

human intuition that struggles to reason about increasingly vast design spaces.

In this work, we propose to combine human insight with automated exploration. We base ATOP on two key ideas: 1) distilling expert design intuitions of past topologies into a unified set of searchable hyperparameters, which concisely cover a rich space of topologies, and 2) searching this hyperparameter space with a scalable multi-objective evolutionary algorithm guided by high-fidelity simulations. This enables the principled discovery of novel high-performance topologies that are flexibly optimized for the unique requirements of each specific workload and cluster, covering metrics like throughput, cost, and reliability.

3 Design of ATOP

We propose ATOP, an Automated Topology Optimization Pipeline, to address the challenges in §1. ATOP consists of three main components: Topology Hyperparameterization Modeling (§3.2), which defines the topology search space; Topology Optimizer (§3.3), which searches and optimizes topologies; and Topology Evaluator (§3.4), which evaluates the given topology. An overview of ATOP is shown in Fig. 4.

3.1 Insights from Existing Topologies

Existing topologies leverage expert knowledge to solve specific problems. Fat-Tree improves aggregation bandwidth in tree topologies, ROFT optimizes inter-server all-reduce in GPU clusters, and HPN enhances ROFT's fault tolerance with a dual-ToR architecture. BCube accelerates one-to-many traffic in modular data centers, while Dragonfly reduces network diameter and cost with low-radix switches. These topologies incorporate hierarchical structures and symmetry, derived from expert experience. We aim to develop a design approach that balances these expert insights with the flexibility of the topology search space to efficiently discover high-performance, deployable topologies that may feature certain asymmetric structures.

To this end, we summarize 11 types of hyperparameters from expert intuitions, constructing a searchable topology space that

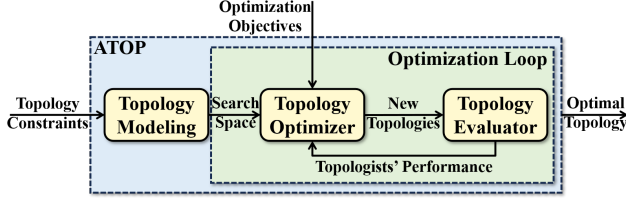


Figure 4: Overview of ATOP

allows the system to explore novel topology designs automatically, not limited to variants or combinations of existing ones. It can produce high-performance asymmetric topologies, such as ZCube (§5), which requires additional ports for middle-layer switches.

3.2 Insight-driven Topology Modeling

ATOP's modeling design is based on an analysis of existing DCN topologies. These topologies typically feature layered structures, symmetry (e.g., HPN, Fat-Tree, BCube), and may include intra-layer connections (e.g., Dragonfly, HyperX, DCell). Thus, ATOP's topology modeling includes two main components: "Node Layering and Inter-layer Connections" and "Intra-layer Connections". Tab. 1 summarizes the notation used in this section.

Node Layering and Inter-layer Connections. Based on observations of classic DCN topologies such as CLOS, Fat-Tree, and BCube, we find that the nodes in these topologies often exhibit a layered structure. For example, the first layer consists of servers, followed by multiple layers of switches. We also discover a certain symmetry in the interconnections between layers in existing topologies. Specifically, the connections between two layers can be divided into multiple blocks of the same size and have the same connection pattern. ATOP's topology modeling follows these design principles.

Firstly, users specify the number of GPUs N_1 and the maximum number of layers $L_{\max}$. The number of switches in the i -th layer is N_i ($2 \leq i \leq L_{\max}$), which is a hyperparameter.

Based on the observation, we model inter-layer connections symmetrically, dividing each layer into blocks. The i -th layer has H_{ij}^i blocks, and the j -th layer has H_{ij}^j blocks, where H_{ij}^i and H_{ij}^j are divisors of N_i and N_j and are also searchable hyperparameters. If $H_{ij}^i = 0$ or $H_{ij}^j = 0$, no interconnection exists. Each block in the i -th layer is connected to one in the j -th layer. Specifically, the k -th block in the i -th layer connects to the $(k \bmod H_{ij}^j + 1)$ -th block in the j -th layer. To ensure connectivity, we set $H_{ij}^i \geq H_{ij}^j$ so that each block in the j -th layer is connected to at least one block in the i -th layer.

The connection pattern between blocks is also defined symmetrically so that each pair of connected blocks between the i -th and j -th layers shares the same connection pattern. Each block in the i -th layer has $\frac{N_j}{H_{ij}^j}$ nodes, and each block in the j -th layer has $\frac{N_i}{H_{ij}^i}$ nodes. For each node in the i -th layer block, there are E_{ij} links to the j -th layer nodes, where $1 \leq E_{ij} \leq \frac{N_j}{H_{ij}^j}$. The k -th node in the i -th layer block connects to the $([E_{ij} \cdot (k - 1) + m] \bmod \frac{N_j}{H_{ij}^j} + 1)$ -th node in the j -th layer. If $E_{ij} = \frac{N_j}{H_{ij}^j}$, the nodes in the two blocks

Category	Type Name	Description
User's Input	$L_{\max}$	The maximum number of layers in the topology.
	N_1	The number of GPUs (first-layer nodes).
	$D_{\max}$	The maximum number of dimensions for nodes in a layer can be divided.
Inter-layer Connections Hyperparameters	N_i	The number of nodes in the i -th layer.
	H_{ij}^i	The number of blocks of the i -th layer in the interconnections between the i -th and j -th layers.
	H_{ij}^j	The number of blocks of the j -th layer in the interconnections between the i -th and j -th layers.
	E_{ij}	The number of links from a i -th layer node to j -th layer nodes in the interconnections between the i -th and j -th layers.
	B_{ij}	The link bandwidth factor for the interconnections between the i -th and j -th layers.
Intra-layer Connections Hyperparameters	D_i	The number of dimensions for nodes in the i -th layer.
	S_i^k	The number of nodes in the k -th dimension of the i -th layer.
	p_i^k	The number of outward connections for nodes in the k -th dimension of the i -th layer.
	A_{rt}^{ik}	The factor for the r -th coordinate of the source node when computing the t -th coordinate of the destination node under the outward connections in the k -th dimension of the i -th layer.
	C_t^{ik}	The bias when computing the t -th coordinate of the destination node under the outward connections in the k -th dimension of the i -th layer.
	B_{ii}	The link bandwidth factor for intra-layer interconnections in the i -th layer.

Table 1: The notations used in topology modeling.

Algorithm 1 Generate hyperparameters for inter-layer connections in ATOP

Input: Maximum number of layer $L_{\max}$; Number of GPUs N_1

Output: Searchable Hyperparameter set $\mathcal{H}$

```

1: for  $i = 2$  to  $L_{\max}$  do
2:   Add  $N_i \in [0, N_1]$  to  $\mathcal{H}$ 
3: end for
4: for  $i = 1$  to  $L_{\max} - 1$  do
5:   for  $j = i + 1$  to  $L_{\max}$  do
6:     Add  $H_{ij}^i \in \{d \mid d > 0, N_i \bmod d = 0\} \cup \{0\}$  to  $\mathcal{H}$ 
7:     Add  $H_{ij}^j \in \{d \mid d > 0, N_j \bmod d = 0\} \cup \{0\}$  to  $\mathcal{H}$ 
8:     Add  $E_{ij} \in [1, \frac{N_j}{H_{ij}^j}]$  to  $\mathcal{H}$ 
9:     Add  $B_{ij} \in [1, 4]$  to  $\mathcal{H}$ 
10:   end for
11: end for
12: return  $\mathcal{H}$ 

```

form a complete bipartite graph. E_{ij} is a hyperparameter determining the connection pattern within the block pairs. An example of inter-layer connection construction is shown in Fig. 5.

Instead of assigning unique bandwidth settings to each link, ATOP configures links in the same inter-layer connection to have consistent bandwidth. Typical RDMA network link bandwidths are 200 Gbps, 400 Gbps, or 800 Gbps. Thus, ATOP uses 200 Gbps as a baseline, scaled by a bandwidth factor. The bandwidth factor B_{ij} , for links between layers i and j , is a hyperparameter ($1 \leq B_{ij} \leq 4$), determining the link bandwidth as $B_{ij} \times 200$ Gbps. This approach accommodates varying link bandwidths within a topology. For example, in HPN [44], the link bandwidth between GPUs and ToR switches is 200 Gbps, while between ToR switches and Aggregate switches it is 400 Gbps.

In summary, the node count per layer N_i , the block count in inter-layer connections H_{ij}^i and H_{ij}^j , the link count between nodes in block pairs E_{ij} , and the bandwidth factor B_{ij} are hyperparameters, as shown in Alg. 1. In addition to the ranges provided in this paper, users can adjust these hyperparameters dynamically based on their requirements, such as fixing or limiting certain hyperparameters.

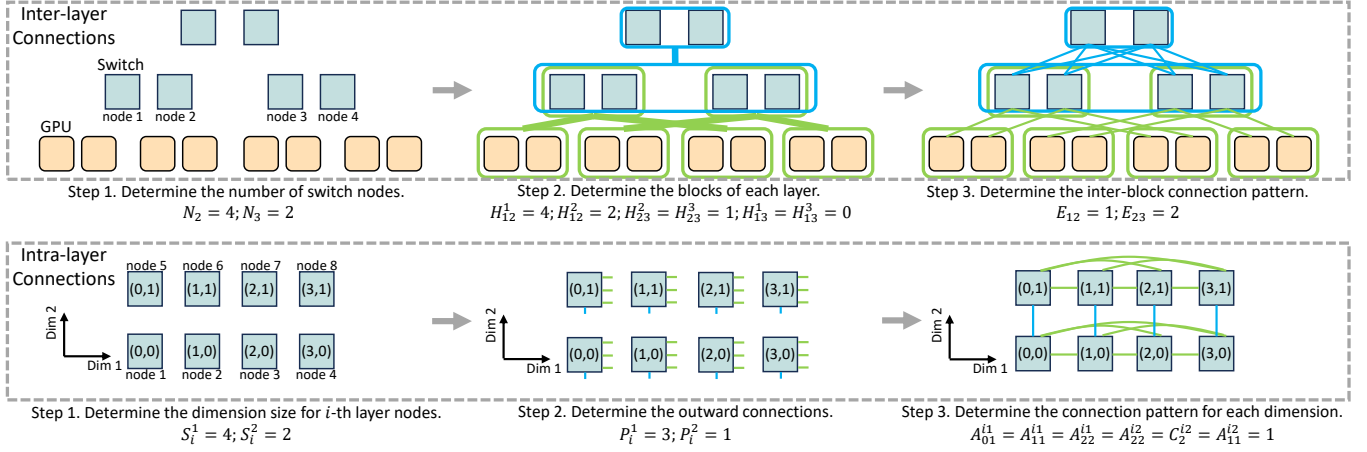


Figure 5: Examples of constructing inter-layer and intra-layer connections in ATOP. Unmentioned hyperparameters = 0.

Intra-layer Connections. In addition to layered architectures, we observe that DCN topologies include dimensional structures and complex connectivity patterns, such as Dragonfly, Torus, and HyperX. To this end, we develop the “intra-layer interconnects” approach integrated into the layered architecture. This method allows direct connections among nodes within the same layer by partitioning the nodes into multiple dimensions. For example, 32 nodes can be divided into two dimensions of 4×8 , with a defined connection pattern for each dimension. Users specify the maximum number of dimensions, denoted as $D_{\max}$.

For the N_i nodes in the i -th layer ($1 \leq i \leq L_{\max}$), they are divided into D_i dimensions ($0 \leq D_i \leq D_{\max}$), where D_i is a searchable hyperparameter. If $D_i = 0$, no intra-layer connections exist for that layer. The number of nodes in each dimension, denoted as S_i^k ($1 \leq k \leq D_i$), is another set of hyperparameters, ensuring that $\prod_{k=1}^{D_i} S_i^k = N_i$. Each node in the layer can be represented by a multi-dimensional coordinate $(X_1, X_2, \dots, X_{D_i})$, where $0 \leq X_1 < S_i^1$, $0 \leq X_2 < S_i^2, \dots, 0 \leq X_{D_i} < S_i^{D_i}$.

Next, we define the number of outward connections for each node in each dimension as a hyperparameter, denoted as P_i^k ($1 \leq k \leq D_i$, $0 \leq P_i^k < S_i^k$). “Outward connections” refers to the number of attempts a node makes to connect to others rather than the node’s degree. Parallel connections are merged and treated as a single connection.

For each node in each dimension, we need to construct connections. A node with coordinates $(X_1, X_2, \dots, X_{D_i})$ forms P_i^k connections in the k -th dimension. The destination of the m -th connection ($1 \leq m \leq P_i^k$) is a node with coordinates $(X'_1, X'_2, \dots, X'_{D_i})$ which can be calculated as:

$$X'_t = [A_{0t}^{ik} \times m + \sum_{r=1}^{D_i} (A_{rt}^{ik} \times X_r) + C_t^{ik}] \bmod S_i^k \quad (1)$$

where $1 \leq t \leq D_i$, A_{rt}^{ik} ($0 \leq r \leq D_i$) and C_t^{ik} are hyperparameters. A_{rt}^{ik} and C_t^{ik} are factors for computing the coordinates of the destination node based on the source node’s coordinates and its different

Algorithm 2 Generate hyperparameters for intra-layer connections in ATOP

Input: Maximum number of dimension $D_{\max}$; Maximum number of layer $L_{\max}$; Number of nodes per layer N_i

Output: Searchable Hyperparameter set $\mathcal{H}$

```

1: for  $i = 1$  to  $L_{\max}$  do
2:   Add  $D_i \in [0, D_{\max}]$  to  $\mathcal{H}$ 
3:    $N_{\text{remained}} \leftarrow N_i$ 
4:   for  $k = 1$  to  $D_i$  do
5:     if  $k \neq D_i$  then
6:       Add  $S_i^k \in \{d \mid N_{\text{remained}} > 0, N_{\text{remained}} \bmod d = 0\}$  to  $\mathcal{H}$ 
7:     else
8:       Add  $S_i^k = N_{\text{remained}}$  to  $\mathcal{H}$ 
9:     end if
10:     $N_{\text{remained}} = N_{\text{remained}} / S_i^k$ 
11:    Add  $P_i^k \in [0, S_i^k - 1]$  to  $\mathcal{H}$ 
12:    for  $t = 1$  to  $D_i$  do
13:      for  $r = 0$  to  $D_i$  do
14:        Add  $A_{rt}^{ik} \in [-S_i^k, S_i^k]$  to  $\mathcal{H}$ 
15:      end for
16:      Add  $C_t^{ik} \in [-S_i^k, S_i^k]$  to  $\mathcal{H}$ 
17:    end for
18:  end for
19:  Add  $B_{ii} \in [1, 4]$  to  $\mathcal{H}$ 
20: end for
21: return  $\mathcal{H}$ 

```

connections, where i represents the interconnections within the i -th layer, k represents the k -th dimension of the outward connection, t denotes the t -th coordinate of the destination node, and r denotes the r -th coordinate of the source node. In other words, the first term accounts for the effects of different connections from the source node, the second term computes the destination coordinates based on each dimension of the source node’s coordinates, and the third is a bias term. The final result ensures that the destination coordinates remain within the given dimensional bounds. This equation is derived from observations of complex intra-layer multi-dimensional interconnect structure. It can provide a high degree of flexibility for intra-layer interconnects, allowing it to model various topological architectures such as multi-dimensional Torus, multi-dimensional FullMesh, Dragonfly, HyperX, and others. Fig. 5 illustrates an example of constructing intra-layer connections. Similar to inter-layer

connections, the intra-layer link bandwidth factor B_{ii} ($1 \leq B_{ii} \leq 4$) is also a hyperparameter.

In summary, the hyperparameters for intra-layer connections include the dimension of each layer D_i , node count per dimension S_i^k , outward connection count P_i^k , coordinate calculation factors A_r^{ik} and C^{ik} , and intra-layer link bandwidth factor B_{ii} , as shown in Alg. 2.

3.3 Topology Optimizer

Optimization Objectives. Optimization objectives are inputs to ATOP and can be customized based on the user's requirements. Our optimization objectives include job completion time (JCT) for representative traffic in LLM training, simulated with a network simulator, ForestColl all-gather [58], average path length under single-switch failure, and network cost. Detailed implementations are provided in §4.1.

Topology Optimizer. The optimizer samples the hyperparameter space, generates a topology for evaluation, and receives performance feedback to guide subsequent sampling. ATOP's optimizer uses NSGA-II as the optimization algorithm due to its performance and efficiency [31, 34, 47]. To improve search efficiency, we also implement parallel optimization, allowing simultaneous multi-topology evaluations with multiple processors, resulting in linear efficiency gains. We also validate ATOP's convergence and efficiency in Appendix D and compare NSGA-II with other optimization algorithms in Appendix J.

3.4 Topology Evaluator

Flow-level Simulator. Efficient evaluation of large-scale topologies is challenging. Packet-level simulators like NS-3 [46] can assess small-scale topologies within minutes but become impractical for thousands of GPUs due to excessive time and memory demands [24]. To address this, we implement a flow-level network simulator that models communication flows between GPUs. The simulator applies the max-min fairness bandwidth allocation [36] algorithm for each flow. For flow collisions, we only use the max-min fairness algorithm to allocate bandwidth and don't use additional penalty functions. For congestion modeling and switch delay, we use the approach from SimGrid [8] for improved accuracy. The delay of each link can be configured in the topology description file. We validate the accuracy of the simulator by comparing its results with NS-3 under the same configurations. Results (Tab. 4 in Appendix I) show the average job completion time error between the two simulators is 1.5% across different scales and topologies in the LLMs training scenario.

2-stage Evaluation. We design a 2-stage evaluation method to reduce the number of time-consuming end-to-end large model training simulations. In the first stage, we evaluate representative fragments of model training workloads, fault tolerance, and network cost as optimization objectives. Due to multi-objective optimization, ATOP can produce a set of Pareto-optimal topologies (defined in Appendix C) as the first-stage results. In the second stage, we evaluate the end-to-end training times for these topologies using Astra-Sim 2.0 [54] with our flow-level network simulator as the backend and SimAI [3] for workload generation. Finally, we obtain end-to-end training results for all Pareto-optimal topologies.

Collective Communication Performance. Although the flow-level simulator evaluates topologies under LLM traffic, existing collective communication algorithms (e.g., Ring [42] and RHD [50]) may not be optimal for a given topology. Therefore, ATOP implements ForestColl's [58] algorithm to compute the theoretical minimum all-gather time. ForestColl's algorithm consists of two parts: part 1 computes the theoretical lower bound on the completion time of an optimal collective algorithm for a given topology, while part 2 derives the actual collective algorithm. While ForestColl can quickly compute the theoretical optimal completion time (part 1), obtaining the concrete collective algorithm and routing (part 2) is time-consuming. Therefore, we only use part 1 to evaluate the topology, serving as one of the optimization objectives during the search process. This evaluation does not use the simulator but relies on a theoretical approach to assess performance under optimal algorithms and ideal load-balancing.

Fault Tolerance. We focus on single-switch failures and propose a theoretical analysis method introducing the metric APL_{fail} , which represents the average path length between GPU pairs under single-switch failure in a topology. The calculation of this metric is as follows: for a network topology $G = (V = V_s \cup V_g, E)$, where V_s denotes the switch nodes and V_g represents the GPUs, APL_{fail} calculates the average communication path length between GPUs under the failure of each switch node and then computes the overall average, formalized as follows:

$$APL_{fail} = \frac{1}{|V_s|} \sum_{v_s \in V_s} \frac{1}{(|V_g|^2 - |V_g|)} \sum_{u, v \in V_g, u \neq v} d_{G'=(V-v_s, E)}(u, v) \quad (2)$$

where $d_{G'=(V-v_s, E)}(u, v)$ represents the length of the shortest path between GPU u and GPU v in the topology after removing the switch v_s .

Network Cost. For the estimation of network costs, we use a real cost model of network hardware. A detailed description is provided in §4.1.

4 Implementation & Evaluation of ATOP

In this section, we discuss the general configurations of ATOP, three use cases, as well as ATOP's search results.

4.1 Methodology & Setup

TestBed. We evaluate ATOP on a single server with 256 AMD EPYC 7Y83 CPU cores and 1 TB of memory.

Search Space. We set L_{max} to 4 as topologies with more layers may have a larger network diameter, resulting in higher latency. We set D_{max} to 4 since most topologies do not exceed four dimensions in intra-layer interconnections.

Optimizer and Objectives. We use NSGA-II for hyperparameter optimization, with 4 types and 14 objectives:

(1) Job Completion Time (JCT) for training traffic patterns (DP-only, PP-only, EP-only, Mixed DP-PP, Mixed DP-EP) using a flow-level network simulator. For non-MoE models, we consider different common parallel strategies. We set TP=8 for the NVIDIA H100 server, which interconnects 8 GPUs with NVLink, and consider PP values of 4, 8, and 16. DP is the number of GPUs divided by TP and PP, using RHD [50] collective communication algorithm. These

traffic patterns generate 9 objectives (3 parallel strategies $\times$ 3 traffic patterns). For MoE models, we consider 256 experts with $EP=256$, DP is the number of GPUs divided by EP, yielding 2 objectives (EP-only, Mixed DP-EP). In total, the above traffic patterns generate 11 optimization objectives.

(2) For ForestColl all-gather, we set the data volume to 1 GB, which is typical for DP in large model training.

(3) For fault tolerance, we use average path length under single-switch failure, as in Equation 2, to estimate topology performance during failures.

(4) For topology cost estimation, we consider switches, cables (copper, fiber cables), optical modules, and NICs. Prices are sourced from FS [19] and Colfax Direct [16]. We use switches with different throughputs: Broadcom N8550-40CD (8 Tbps), N9550-32D (12.8 Tbps), N9550-64D (25.6 Tbps), and N9600-64OD (51.2 Tbps). NICs include NVIDIA ConnectX-6 and ConnectX-7, and optical modules are NVIDIA OSFP Optical Modules series based on bandwidth. Cable selection depends on distance: Direct Attach Cable (DAC) for short-range (≤ 3 meters), optical modules and MTP jumpers (fiber cables) for long-range. For NIC-to-first-layer switch interconnections, if a rail-optimized method is used (i.e., NICs on the same server connect to different switches), we assume 15-meter cables with MTP jumpers and optical modules. If all NICs on the same server connect to the same ToR, we use 3-meter DACs. For adjacent-layer switches, we use 30-meter MTP jumpers, and for cross-layer (e.g., layer-1 to layer-3) and intra-layer connections, we use 50-meter MTP jumpers.

Number of Topology Searches and Search Duration. Despite a reduced search space, evaluating all topologies remains infeasible. We limit the optimizer to explore up to 10^5 topologies. ATOP completes the search and 2-stage evaluation in 6.5 hours for 256 GPUs, 10.6 hours for 1k GPUs, 25.4 hours for 4k GPUs, and 71.2 hours for 16k GPUs, with 80% of the time spent on topology evaluation. We have already parallelized the search and performed the search process with 256 CPU cores. These times are reasonable compared to the months required for data center deployment. Moreover, metrics like HyperVolume [60] can also serve as stopping criteria. We also discuss ATOP's convergence in Fig. 16 of Appendix D. The results demonstrate that ATOP shows convergence across three metrics.

Second Stage Evaluation Setup. In the second evaluation stage, we conduct end-to-end large model training simulations for Pareto-optimal topologies using Astra-Sim 2.0. We evaluate topologies for GPT-3-22B training on 256 GPUs and GPT-3-175B training on 1k, 4k, and 16k GPUs. For parallel strategies, we adopt common configurations [4, 11, 35, 56]: $TP=8$, $PP=8$, and DP as the number of GPUs divided by TP and PP, with the interleaved 1F1B strategy. We use intra-server interconnects for TP communication and inter-server interconnects for DP and PP. We arrange the parallel strategy consistent with Megatron-LM [35], which makes efforts to put a set of tightly-coupled GPUs more closely together to minimize switch hops. We record the computation time for each model's operator on the NVIDIA H100 GPU. Detailed model parameters and training configurations are provided in Tab. 8 in the Appendix. The 2-stage evaluation method avoids 10^5 full-scale model training simulations, reducing evaluation to approximately 5000 (5%) Pareto-optimal topologies and achieving a 20-fold speedup.

Topology Evaluator and Simulation Configuration. In the flow-level network simulator, the baseline inter-server link bandwidth is set to 200 Gbps, adjustable via the bandwidth factor. Link delay is 5 μ s, with the total latency of each flow dependent on hop count. For 8-GPU NVIDIA H100 servers, intra-server GPUs are interconnected via NVLink and NVSwitch with 900 GB/s bidirectional bandwidth and 1 μ s latency, consistent with the NVIDIA DGX H100 server [39] specifications. During simulation evaluation, PXN is enabled and routes traffic via NVLink to minimize hop count when beneficial. ECMP [28] is employed for routing and load balancing, chosen for its deployability as the de-facto standard in commercial RoCE switches [20–22]. Topologies performing well under ideal load-balancing but poorly under ECMP are difficult to deploy directly using commercial RoCE switches. Considering deployability and simulation efficiency, we use ECMP for evaluation. For fault tolerance, intra-server interconnection is considered, and PXN is also enabled. We treat each 8-GPU server as having point-to-point connections among GPUs, with a communication path length of 1. Since Pareto-optimal topologies are selected from all generated by ATOP, a topology that disconnects after a ToR switch failure but excels in other metrics may still be chosen, ensuring ATOP does not generate only multi-ToR topologies.

4.2 Use Cases of ATOP

Next, we show that ATOP is useful in the three operational scenarios in §1.

4.2.1 Use Case 1: Designing a New Data Center.

Setup. In designing a new data center, ATOP is employed to explore topologies at various scales: 256, 1024, 4096, and 16384 GPUs. Switch throughput limits are set to 6.4 Tbps for 256 GPUs, 12.8 Tbps for 1024 GPUs, 25.6 Tbps for 4096 GPUs, and 51.2 Tbps for 16384 GPUs. For NICs, we consider configurations with multiple NICs per GPU, including multi-port NICs, with each GPU accommodating up to 8 NIC ports, each with at least 200 Gbps bandwidth. The maximum outbound bandwidth per GPU is capped at 1.6 Tbps. A GPU with 400 Gbps outbound bandwidth can utilize either a single-port 400 Gbps NIC or a dual-port 2×200 Gbps NIC. ATOP can freely explore NIC port counts and bandwidth configurations for each GPU.

Results. Fig. 1 visualizes the search results for data centers at various scales. Since the first-stage evaluation uses 14 optimization objectives, we only show end-to-end LLM training evaluation results of Pareto-optimal topologies. Each point represents a topology. ATOP identifies the Pareto frontier for training time and cost, with Pareto-optimal topologies positioned near this frontier. A comparison of 100,000 ATOP-discovered topologies and Pareto-optimal topologies for the 4k-GPU scale is shown in Fig. 6. Results show that ATOP captures topologies on the performance-cost Pareto frontier, validating the feasibility of using representative traffic patterns during training to evaluate topologies.

From these results, we observe the following pattern: performance improves with increased cost within a range, but further cost escalation yields diminishing returns once a performance threshold is reached.

We identify three key topologies for each GPU scale: the most cost-effective, best-performance, and budget-friendly topologies.

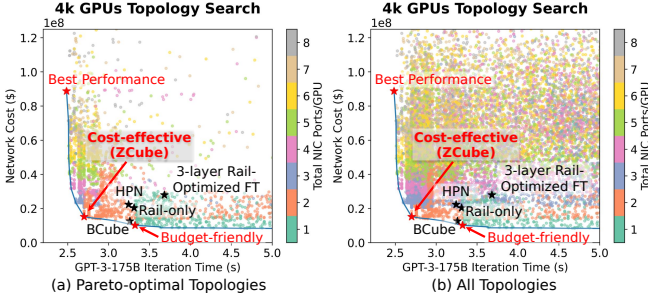


Figure 6: During the 4k GPUs search process: (a) The Pareto-optimal topologies generated by ATOP; (b) All the topologies generated by ATOP.

The most cost-effective topology is located at the Pareto frontier’s inflection point. The best-performance topology has minimal iteration time. The budget-friendly topology delivers strong performance at a lower cost by configuring only one NIC port per GPU. Notably, the most cost-effective topology at the inflection point across all scales shares a similar architecture. This topology, named ZCube, is selected by our users for small-scale deployment and testing. A formal description and analysis of ZCube across different GPU scales are provided in §5 and §6.

4.2.2 Use Case 2: Adjusting an Existing Data Center.

Setup. In this scenario, we use ATOP to optimize the topology of a 4k GPU data center. The initial topology is a 3-layer non-blocking ROFT architecture. We aim to improve performance while reducing network costs by re-wiring without purchasing additional switches and NICs. Modifications to optical modules and cables are allowed. The switch throughput is 25.6 Tbps, the number of switches is 320, and each GPU can use a dual-port NIC. These constraints strictly limit the topology search space.

Results. Fig. 7(a) presents the ATOP search results in this scenario. We discover that ZCube is both the best-performance and cost-effective topology within this constrained search space. The strict limitations on the number of switches and throughput make constructing topologies with higher performance difficult. ATOP also identifies the Pareto frontier of training time and network cost in this scenario. Topologies on the Pareto frontier represent potential options from which users can select based on their budget.

4.2.3 Use Case 3: Expansion.

Setup. In this scenario, we consider expanding an existing 1k GPU data center to a 4k GPU data center. The initial topology is a 2-layer non-blocking ROFT, using switches with 25.6 Tbps throughput, with each GPU configured with a dual-port NIC. In ATOP, the expanded topology should retain the original switches and NICs. The throughput of newly added switches is limited to 51.2 Tbps. Each GPU is restricted to using one dual-port NIC. In addition, we introduce a new optimization objective to measure the difference between the expanded topology $G_e = (V_e, E_e)$ and the original topology $G_o = (V_o, E_o)$. Specifically, we use the number of modified links as a metric, which is defined as follows: $M_{\text{Link}} = |E_e| + |E_o| - 2|E_e \cap E_o|$. We aim to minimize this objective to reduce workforce costs. We don’t use this metric in Case 2 because it would result in a zero value, meaning no modification to the topology, which would

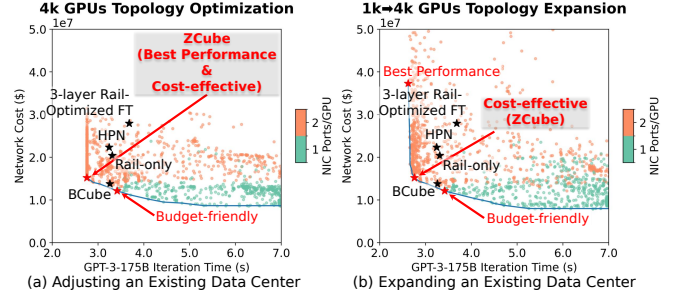


Figure 7: (a) The search results of ATOP when adjusting an existing 4k-GPU DCN. (b) The search results when expanding a DCN from 1k GPUs to 4k GPUs.

be unfair to other topologies. However, in Case 3, even expanding directly to a 3-layer ROFT requires adding some links.

Results. Fig. 7(b) presents the search results of ATOP in this scenario, comparing network cost and training performance. We also compare the number of modified links (another cost metric) and training performance in Fig. 13 in the Appendix. Results show that ATOP identifies the Pareto frontier in both cost metrics. We also discover that ZCube remains the most cost-effective topology.

4.3 Discussion

ATOP is a useful tool for data center architects and is not limited to the above three applications. Evolving hardware configurations, shifting traffic patterns, and dynamic user requirements often necessitate the discovery of novel topologies. By adjusting its inputs, ATOP can dynamically adapt to these changes and efficiently identify optimized topologies that meet updated constraints. For example, in Appendix A & B, we show that ATOP can also produce new and high-quality topologies for a multi-tenant data center and a heterogeneous data center, respectively.

From the results of ATOP, we observe that the topologies at the inflection point (cost-effective) share similar architectures. Most of them have two or three layers of switches, with connections between GPUs and switches at each layer, as well as connections between the switches. Both users and we found this topology to be of significant interest and proceeded with small-scale deployments (§6.2). We summarize and define this type of topology as ZCube. A formal description and analysis of ZCube across different GPU scales are provided in §5 and §6.

5 ZCube: A New Topology from ATOP

In this section, we first formalize the construction of ZCube with mathematical definitions in §5.1. Then, we analyze the properties of ZCube in §5.2, including scalability and diameter. Regarding its fault tolerance, we provide a detailed analysis in Appendix G.

5.1 ZCube Construction

The construction of ZCube employs a recursive method. The smallest unit, $\text{ZCube}(n, 1)$, is a single switch connecting n GPUs. Generally, a $\text{ZCube}(n, k+1)$ ($k \geq 1$) is constructed from n $\text{ZCube}(n, k)$ and n^k switches. Thus, $\text{ZCube}(n, k+1)$ has $N = n^{k+1}$ GPUs and $k+1$ levels of switches, with n^k switches per level. Each GPU in a

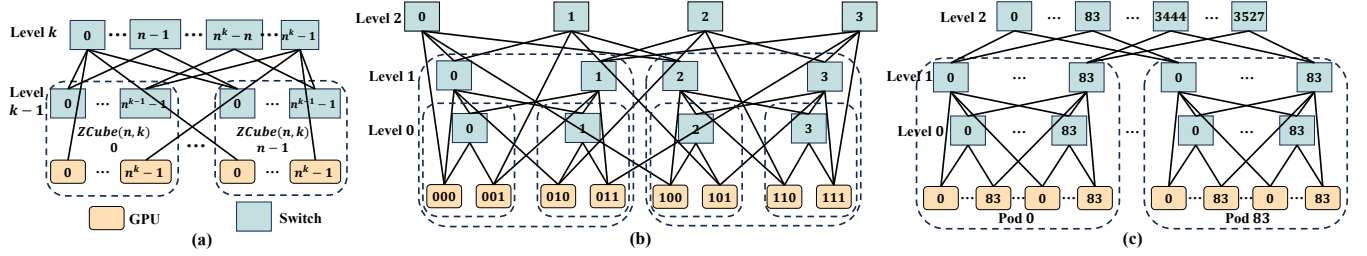


Figure 8: (a) A $ZCube(n, k+1)$ is constructed from n $ZCube(n, k)$ and n^k switches. (b) An example of $ZCube(2, 3)$. (c) An example of $ZCube(84, 3)$ -partial.

$ZCube(n, k+1)$ is equipped with $(k+1)$ NIC ports numbered from level-0 to level- k . In addition to equipping each GPU with multi-port NICs [41], $ZCube$ can also be implemented using multiple NICs for each GPU. For instance, when constructing $ZCube(n, 2)$, each GPU only needs to be equipped with a 400 Gbps NIC, which is then divided into 2×200 Gbps ports and connected to different switches, or alternatively, using two separate 200 Gbps NICs.

The connections between switches and GPUs in $ZCube(n, k+1)$ can be constructed as follows. We number the n $ZCube(n, k)$ from 0 to $n-1$ and the GPUs in each $ZCube(n, k)$ from 0 to n^k-1 . We then connect the level- k NIC of the i -th GPU ($i \in [0, n^k-1]$) in the j -th $ZCube(n, k)$ ($j \in [0, n-1]$) to the j -th port of the i -th level- k switch.

For the connections between switches, each level- $(k-1)$ switch connects to n level- k switches. The i -th level- $(k-1)$ switch ($i \in [0, n^{k-1}-1]$) in the j -th $ZCube(n, k)$ ($j \in [0, n-1]$) connects to n level- k switches, represented as the m -th switch ($m \in [i \cdot n, (i+1) \cdot n-1]$) in level- k , as illustrated in Fig. 8(a). We also present a schematic of $ZCube(2, 3)$ in Fig. 8(b).

In $ZCube(n, k)$, level-0 and level- $(k-1)$ switches require $2n$ ports, while switches at the intermediate levels require $3n$ ports, highlighting the asymmetry of $ZCube$. In traditional topologies such as Fat-Tree, BCube, and Dragonfly, switches typically use the same number of ports, reflecting a preference for symmetry. This focus on symmetry may lead designers to overlook asymmetrical yet high-performance topologies. Using ATOP can help designers explore asymmetrical topologies.

5.2 Properties of ZCube

Scalability and Diameter. For a network, the *diameter* is the largest hop count of the shortest path among all GPU pairs. In $ZCube(n, 2)$, each GPU has 2 NIC ports and switches with $2n$ ports can connect n^2 GPUs. A $ZCube(128, 2)$ topology using 256-port switches interconnects 16384 GPUs with a network diameter of only 2. In $ZCube(n, 4)$, each GPU has 4 NIC ports and switches with $3n$ ports connect n^4 GPUs. A $ZCube(42, 4)$ topology using 128-port switches supports 3,111,696 GPUs with a network diameter of 4. By contrast, a 3-layer Fat-Tree topology with 128-port switches connects only 524,288 GPUs (16.8%) with a network diameter of 5.

For larger GPU clusters constrained by dual-port NICs, multiple $ZCube(n, 2)$ topologies can be combined through core switches. For example, n $ZCube(n, 2)$ topologies connected via $n^2/2$ level-2 switches form a CLOS topology, resulting in a $ZCube(n, 3)$ -partial

Topology	Max Hop Count
3-layer Rail-Optimized FT	5
2-layer Rail-only	3
2-layer HPN	3
BCube($n, 2$)	3
BCube($n, 3$)	5
3D-Torus	$\lceil 3/2 \sqrt[3]{N} \rceil$
Dragonfly	4
SlimFly [6]	3
$ZCube(n, 2)$	2
$ZCube(n, 3)$	3
$ZCube(n, 3)$ -partial	4
$ZCube(n, 4)$	4

Table 2: Network diameter of other topologies. N represents the number of endpoints.

topology. This design omits direct connections between level-2 switches and GPUs. Using 256-port switches, a $ZCube(84, 3)$ -partial topology (shown in Fig. 8(c)) interconnects 592,704 GPUs with a network diameter of 4. It includes 84 $ZCube(84, 2)$ pods, each containing 7056 GPUs with a network diameter of 2.

Theorem 1. *The network diameter of a $ZCube(n, k)$ is k .*

Theorem 2. *The diameter of $ZCube(n, 3)$ -partial is 4.*

We provide the proof of the theorems in Appendix E. The comparison with other topologies is illustrated in Tab. 2.

Hardware Cost Estimate. When connecting GPUs at the same scale, $ZCube$ requires fewer switches and lower-bandwidth cables (e.g., $ZCube$'s 200G links vs. ROFT's 400G links) compared to ROFT, Rail-only, and HPN. We present topology diagrams for building a 16384-GPU cluster using 51.2 Tbps switches under different topologies, along with the corresponding switch and cable requirements in Appendix K. All servers are configured with 8 GPUs and 8×400 G NICs, which is one of the most common setups [39]. All ports of all switches are fully utilized to minimize the number of switches as much as possible. Compared to ROFT, Rail-only, and HPN, $ZCube$ reduces switch count by 33% to 60% and optical transceiver costs by 25% to 50%. This yields substantial cost reductions for DCN builders. Detailed data are provided in Appendix K&L.

Fault Tolerance. $ZCube$ demonstrates strong fault tolerance as shown in Fig. 3(b). In addition, compared to ROFT, HPN, and Rail-only, $ZCube$ uses fewer switches (fewer failure points) and therefore has a lower probability of failure. Based on observations from real

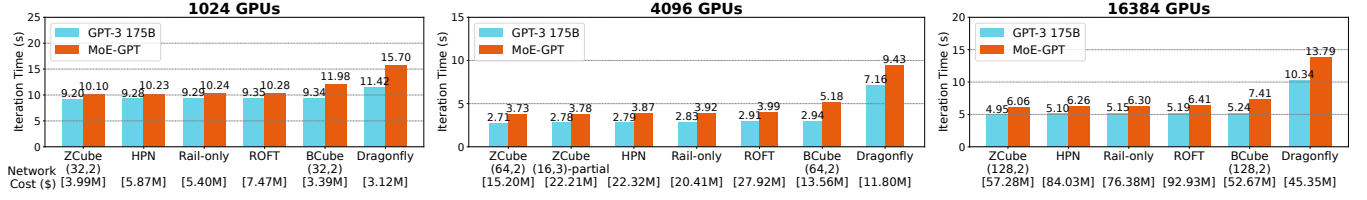


Figure 9: The training iteration time for GPT-3 175B and MoE-GPT models and the corresponding network costs on various topologies, under different numbers of GPUs.

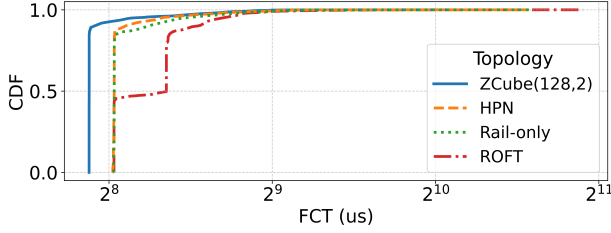


Figure 10: CDF of PP flow completion time during a GPT-3 175B training iteration on 16384 GPUs.

GPU clusters, the failure probability of a single switch is 0.03%. For a 16384-GPU cluster, the probability of no switch failure at the same time is 93% for ZCube, 89% for HPN and Rail-only, and 83% for ROFT. We also discuss two common network failure scenarios (single-switch failures and link failures) in Appendix G. ZCube shows robustness in both scenarios based on theoretical analysis and simulation experiments.

6 Evaluation of ZCube

We evaluate the ZCube’s performance and compare it with various topologies at different scales. In §6.1, to evaluate the performance of these topologies under packet-spraying [17] load balancing, we adopt a packet-level simulator (htsim [7]) as the backend of Astra-Sim 2.0 [54]. We measure the end-to-end iteration time for training GPT-3 175B and MoE-GPT models. We also evaluate the theoretical ForestColl all-gather performance of the topologies in §6.1, which shows these topologies have identical theoretical ForestColl all-gather performance. In §6.2, we deploy real-world ZCube(4, 2) and ROFT on 4 servers with 16 H800 GPUs and test the performance of NCCL.

6.1 End-to-end LLMs Training Performance

We evaluate the training iteration time of LLMs across various topologies at scales of 1024, 4096, and 16384 GPUs (with topology configurations detailed in Appendix L). All topologies share the same server configuration: each server has 8 H100 GPUs connected via 900 GB/s NVLink. Each server is equipped with 8 NICs, providing a total outbound bandwidth of 3.2 Tbps. We deploy two models: GPT-3 175B and MoE-GPT. For GPT-3 175B training, the parallel strategy is consistent with §4.1. For MoE-GPT, we follow the configuration in DeepSpeed-MoE [45]. We set 256 experts with EP=256. The DP of the MoE layer is the number of GPUs divided by the EP. For non-MoE layers, we set TP=8, and the DP is the number of

GPUs divided by the TP. Other configurations remain consistent with GPT-3 175B training.

In all evaluations, we enable PXN and ensure that all traffic is rail-aligned. In the packet-level network simulator—htsim [7], we integrate Broadcom’s Tomahawk5 RoCE switch model and enable the Dynamic Load Balancing (DLB) feature, a *packet-spraying mechanism* that dynamically selects a port for each packet based on factors such as port status, queue depth, and latency. In Appendix H, we compare the collective communication results between the simulator and the real-world testbed in §6.2, the results show an average relative error of only 5%.

As shown in Fig. 9, ZCube demonstrates the best performance in training both non-MoE and MoE models. Compared to ROFT, Rail-only, and HPN, ZCube improves end-to-end LLM training time by 3% to 7% and reduces network hardware costs by 26% to 46%. This acceleration comes from ZCube’s low-diameter property. For example, Fig. 10 shows the flow completion time CDF of PP traffic. In ROFT, Rail-only, and HPN, PP traffic typically requires 3 or even 5 hops, while in ZCube it only requires 2 hops. In Appendix K, we present the topology diagrams of ROFT, Rail-only, HPN, and ZCube under a 16384 GPU scale, highlighting the cost-effectiveness of ZCube.

6.2 Building and Testing a Real ZCube

We deploy the ZCube topology in a real-world setting and conduct NCCL-based collective communication tests (without PP), confirming its superior cost-effectiveness.

Our testbed consists of 4 servers and 8 Mellanox QM9790 InfiniBand switches [37]. Each server has 8 NVIDIA H800 GPUs. GPUs in the same server are interconnected with 200 GB/s NVLink. We use 16 NVIDIA ConnectX-7 single-port 400GbE NICs [38] to build the ROFT topology and 16 ConnectX-7 dual-port 2×200GbE NICs [38] for the ZCube topology, as shown in Fig. 11. To avoid NIC bandwidth contention between GPUs during testing, we only use 4 GPUs per server, totaling 16 GPUs for the collective communication tests. In this configuration, each GPU in both ZCube and ROFT has 400 Gbps of outbound bandwidth, and each server has a total outbound bandwidth of 1.6 Tbps. For routing and load balancing, we use hand-crafted static optimal routing to eliminate flow collisions.

We evaluate their performance using NCCL 2.21.5 [42] and nccl-tests [40]. We use the default collective communication algorithms in NCCL. Fig. 12 presents the results. For each data size, we repeat the test 100 times and report the mean and standard deviation.

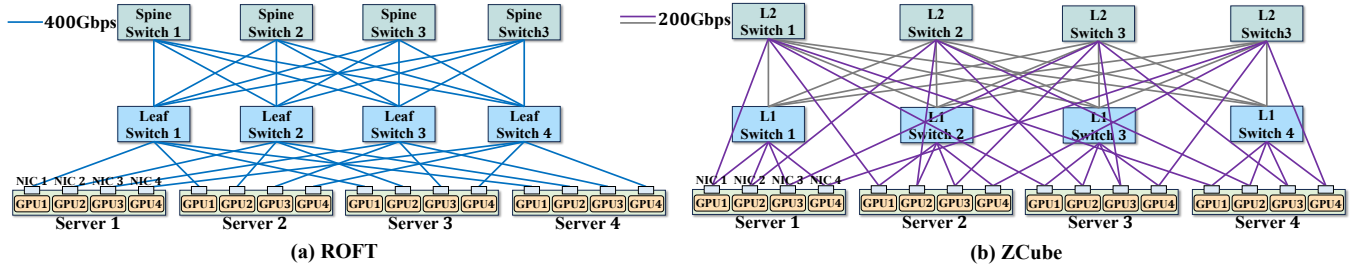


Figure 11: The topology diagrams of ROFT and ZCube on a real testbed.

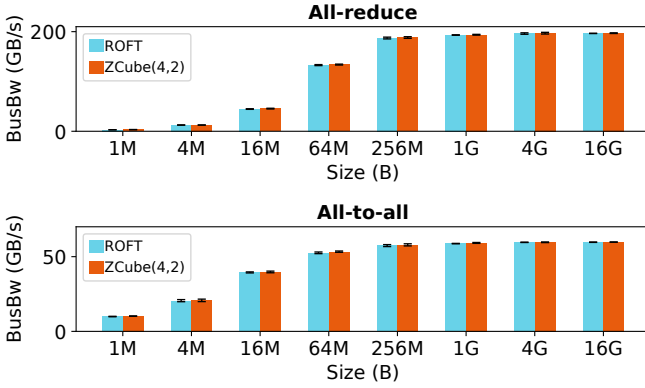


Figure 12: Collective communication performance on real-world deployment.

ZCube and ROFT achieve the same all-reduce and all-to-all performance, while ZCube reduces hardware cost by 25% by using only 48×200G links, compared to ROFT’s 32×400G links.

In summary, compared to state-of-the-art topologies such as ROFT, Rail-only, and HPN, ZCube achieves the same ideal bandwidth performance (as shown by the same theoretical ForestColl all-gather performance and Fig. 12), while offering lower network cost (Fig. 9 and Appendix K). Its lower network diameter (Tab. 2) accelerates PP flow, thereby improving end-to-end large model training performance (Fig. 9 and Fig. 10). ZCube also provides strong fault tolerance (Fig. 3(b), §5.2 and Appendix G).

7 Related Work

DCN Topology. The Rail-Optimized topology [39] is commonly used in large-scale clusters but suffers from poor fault tolerance and high cost. Rail-only reduces cost but still suffers from limited fault tolerance. HPN [44] addresses the fault tolerance issue with a dual-ToR structure; however, this design increases network costs due to the need for additional links. HammingMesh [27] reduces network costs by designing a topology based on existing deep-learning traffic patterns. However, changes in model architecture may introduce more complex traffic, failing to improve all-to-all performance. SlimFly [6] designs a low-diameter network topology based on MMS graph theory.

Topology Design Method. While various approaches currently assist network designers in topology design, large-scale automated

topology design and optimization remain unexplored. Condor [48] introduces a topology description language for modeling but requires manual design for new topologies. Yijia Chang et al. [10] propose a unified topology modeling framework. However, these works still rely on expert experience and do not support performance-oriented optimization, making them not directly comparable with ATOP. TopoOpt [52] presents a joint optimization method for topology and parallel strategies, but it is limited to small-scale direct-connect topologies and requires optical switches. Liangyu Zhao et al. [59] focus on optimizing direct-connect topologies and collective communication. These works require optical switches and have limited scalability, whereas ATOP aims to address the problem of electrical switch-based DCN topology design. Therefore, we do not compare these approaches to ATOP in the evaluation.

8 Conclusion

In this paper, we propose an automated topology optimization pipeline, ATOP, capable of identifying the superior large-scale data center network topology in a short time. ATOP supports the simultaneous optimization of multiple objectives, including network topology performance, hardware cost, and fault tolerance. ATOP also supports various topology design scenarios, such as building, optimizing, and expanding data centers. Using ATOP, we discover ZCube, a new topology with superior cost-effectiveness, low network diameter, and strong fault tolerance. Simulation experiments show that ZCube improves end-to-end LLM training speed by 3% to 7% and reduces network hardware costs by 26% to 46% compared to previous state-of-the-art topologies, including Rail-optimized Fat-tree (ROFT), Rail-only, and HPN. We also construct ZCube on a real-world testbed. Results show that ZCube reduces hardware costs by 25% compared to ROFT while maintaining the same all-reduce and all-to-all performance.

ACKNOWLEDGEMENT

We thank our shepherd Dr. Manya Ghobadi, the anonymous SIGCOMM reviewers, and Weiyang Wang for their constructive comments. We also thank Weirong Jiang, Lei Wang, and Lulu Chen from ByteDance for their assistance in the development of the network simulator and the real-world testbed. Prof. Dan Li and Dr. Li Chen are the corresponding authors. This work was supported by the National Key R&D Program of China (2022YFB3105000), the Beijing Outstanding Young Scientist Program (No. JWZQ20240101008), and Zhongguancun Laboratory.

References

- [1] Jung Ho Ahn, Nathan Binkert, Al Davis, Moray McLaren, and Robert S Schreiber. 2009. HyperX: topology, routing, and packaging of efficient large-scale networks. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*. 1–11.
- [2] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. 2008. A scalable, commodity data center network architecture. *ACM SIGCOMM computer communication review* 38, 4 (2008), 63–74.
- [3] Alibaba. 2024. SimAI. <https://github.com/aliyun/SimAI>
- [4] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, M rouane Debbah,  tienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, et al. 2023. The falcon series of open language models. *arXiv preprint arXiv:2311.16867* (2023).
- [5] Syrine Belakaria, Aryan Deshwal, and Janardhan Rao Doppa. 2019. Max-value entropy search for multi-objective Bayesian optimization. *Advances in neural information processing systems* 32 (2019).
- [6] Maciej Besta and Torsten Hoefler. 2014. Slim fly: A cost effective low-diameter network topology. In *SC'14: proceedings of the international conference for high performance computing, networking, storage and analysis*. IEEE, 348–359.
- [7] Broadcom. 2023. htsim. <https://github.com/Broadcom/csg-htsim>
- [8] Henri Casanova, Arnaud Giersch, Arnaud Legrand, Martin Quinson, and Fr d ric Suter. 2014. Versatile, Scalable, and Accurate Simulation of Distributed Applications and Platforms. *J. Parallel and Distrib. Comput.* 74, 10 (June 2014), 2899–2917. <http://hal.inria.fr/hal-01017319>
- [9] Yair Censor. 1977. Pareto optimality in multiobjective problems. *Applied Mathematics and Optimization* 4, 1 (1977), 41–59.
- [10] Yijia Chang, Xi Huang, Longxiulin Deng, Ziyu Shao, and Junshan Zhang. 2020. Systematic topology design for large-scale networks: A unified framework. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 347–356.
- [11] Bo Chen, Xingyi Cheng, Pan Li, Yangli-ao Geng, Jing Gong, Shen Li, Zhilei Bei, Xu Tan, Boyan Wang, Xin Zeng, et al. 2024. xTrimoPGLM: unified 100B-scale pre-trained transformer for deciphering the language of protein. *arXiv preprint arXiv:2401.06199* (2024).
- [12] Seung-Seok Choi, Sung-Hyuk Cha, Charles C Tappert, et al. 2010. A survey of binary similarity and distance measures. *Journal of systemics, cybernetics and informatics* 8, 1 (2010), 43–48.
- [13] Charles Clos. 1953. A study of non-blocking switching networks. *Bell System Technical Journal* 32, 2 (1953), 406–424.
- [14] Andrew R Curtis, Tommy Carpenter, Mustafa Elsheikh, Alejandro L pez-Ortiz, and Srinivasan Keshav. 2012. Rewire: An optimization-based framework for unstructured data center network design. In *2012 Proceedings IEEE INFOCOM*. IEEE, 1116–1124.
- [15] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* 6, 2 (2002), 182–197.
- [16] Colfax Direct. 2024. <https://www.colfaxdirect.com>
- [17] Advait Dixit, Pawan Prakash, Y Charlie Hu, and Ramana Rao Kompella. 2013. On the impact of packet spraying in data center networks. In *2013 proceedings ieee infocom*. IEEE, 2130–2138.
- [18] Elon Musk. 2024. Colossus Cluster. <https://x.com/elonmusk/status/1830650370336473253>
- [19] FS. 2024. <https://www.fs.com/>
- [20] FS. 2024. N8550-24CD8D, 24-Port Ethernet L3 Data Center Switch. https://www.fs.com/products/207079.html?now_cid=4369
- [21] FS. 2024. N9510-64D, 64-Port Ethernet L3 Data Center Switch. https://www.fs.com/products/149853.html?now_cid=3255
- [22] FS. 2024. N9600-128QC, 128-Port Ethernet L3 Data Center Switch. https://www.fs.com/products/241601.html?now_cid=3255
- [23] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 57–70.
- [24] Kaihui Gao, Li Chen, Dan Li, Vincent Liu, Xizheng Wang, Ran Zhang, and Lu Lu. 2023. Dons: Fast and affordable discrete event network simulation with automatic parallelization. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 167–181.
- [25] Chuanxiong Guo, Guohan Lu, Dan Li, Haitao Wu, Xuan Zhang, Yunfeng Shi, Chen Tian, Yongguang Zhang, and Songwu Lu. 2009. BCube: a high performance, server-centric network architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2009 conference on Data communication*. 63–74.
- [26] Chuanxiong Guo, Haitao Wu, Kun Tan, Lei Shi, Yongguang Zhang, and Songwu Lu. 2008. Dcell: a scalable and fault-tolerant network structure for data centers. In *Proceedings of the ACM SIGCOMM 2008 conference on Data communication*. 75–86.
- [27] Torsten Hoefler, Tommaso Bonato, Daniele De Sensi, Salvatore Di Girolamo, Shigang Li, Marco Heddes, Jon Belk, Deepak Goel, Miguel Castro, and Steve Scott. 2022. HammingMesh: a network topology for large-scale deep learning. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–18.
- [28] Christian Hopps. 2000. *Analysis of an equal-cost multi-path algorithm*. Technical Report.
- [29] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. 2024. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 745–760.
- [30] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–14.
- [31] Florian Karl, Tobias Pielok, Julia Moosbauer, Florian Pfisterer, Stefan Coors, Martin Binder, Lennart Schneider, Janek Thomas, Jakob Richter, Michel Lang, et al. 2023. Multi-objective hyperparameter optimization in machine learning—An overview. *ACM Transactions on Evolutionary Learning and Optimization* 3, 4 (2023), 1–50.
- [32] John Kim, William J Dally, Steve Scott, and Dennis Abts. 2008. Technology-driven, highly-scalable dragonfly topology. *ACM SIGARCH Computer Architecture News* 36, 3 (2008), 77–88.
- [33] Juncal Liu, Jessie Hui Wang, and Yimin Jiang. 2023. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 486–498.
- [34] Alejandro Morales-Hern ndez, Inneke Van Nieuwenhuysse, and Sebastian Rojas Gonzalez. 2023. A survey on multi-objective hyperparameter optimization algorithms for machine learning. *Artificial Intelligence Review* 56, 8 (2023), 8043–8093.
- [35] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [36] Anh Tuan Nguyen and Frank Eliassen. 2009. An efficient solution for max-min fair rate allocation in p2p simulation. In *2009 International Conference on Ultra Modern Telecommunications & Workshops*. IEEE, 1–5.
- [37] NVIDIA. 2022. Mellanox QM9790 InfiniBand Switch. <https://docs.nvidia.com/networking/display/qm9700-and-qm9790-1u-ndr-400gbps-infiniband-switch-systems-user-manual.pdf>
- [38] NVIDIA. 2022. NVIDIA ConnectX-7 400G Adapters Accelerated Networking for Modern Data Center Infrastructures. <https://resources.nvidia.com/en-us-accelerated-networking-resource-library/connectx-7-datasheet>
- [39] NVIDIA. 2023. SuperPOD: Next Generation Scalable Infrastructure for AI Leadership. <https://docs.nvidia.com/https://docs.nvidia.com/dgx-superpod-reference-architecture-dgx-h100.pdf>
- [40] NVIDIA. 2024. nccl-tests. <https://github.com/NVIDIA/nccl-tests/>
- [41] NVIDIA. 2024. NVIDIA Bluefield-3 DPU Programmable Data Center Infrastructure On-a-chip. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-3-dpu.pdf>
- [42] NVIDIA. 2024. NVIDIA Collective Communications Library (NCCL). <https://developer.nvidia.com/nccl>
- [43] Art B Owen. 2000. Monte Carlo, quasi-Monte carlo, and randomized quasi-Monte Carlo. In *Monte-Carlo and Quasi-Monte Carlo Methods 1998: Proceedings of a Conference held at the Claremont Graduate University, Claremont, California, USA, June 22–26, 1998*. Springer, 86–97.
- [44] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 486–498.
- [45] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.
- [46] George F Riley and Thomas R Henderson. 2010. The ns-3 network simulator. In *Modeling and tools for network simulation*. Springer, 15–34.
- [47] Anthony Sarah, Sharath Nittur Sridhar, Maciej Szankin, and Sairam Sundaresan. 2024. LLaMA-NAS: Efficient Neural Architecture Search for Large Language Models. *arXiv preprint arXiv:2405.18377* (2024).
- [48] Brandon Schl inker, Radhika Niranjan Mysore, Sean Smith, Jeffrey C Mogul, Amin Vahdat, Minlan Yu, Ethan Katz-Bassett, and Michael Rubin. 2015. Condor: Better topologies through declarative design. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 449–463.
- [49] Siddharth Singh, Olatunji Ruwase, Ammar Ahmad Awan, Samyam Rajbhandari, Yuxiong He, and Abhinav Bhatle. 2023. A hybrid tensor-expert-data parallelism approach to optimize mixture-of-experts training. In *Proceedings of the 37th*

- International Conference on Supercomputing*. 203–214.
- [50] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. 2005. Optimization of collective communication operations in MPICH. *The International Journal of High Performance Computing Applications* 19, 1 (2005), 49–66.
 - [51] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, Ying Zhang, and Naader Hasani. 2024. Rail-only: A low-cost high-performance network for training llms with trillion parameters. In *2024 IEEE Symposium on High-Performance Interconnects (HOTI)*. IEEE, 1–10.
 - [52] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Manya Ghobadi, Zhihao Jia, Dheevatsa Mudigere, Ying Zhang, and Anthony Kewitsch. 2023. {TopoOpt}: Co-optimizing Network Topology and Parallelization Strategy for Distributed Training Jobs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 739–767.
 - [53] Tianwen Wei, Bo Zhu, Liang Zhao, Cheng Cheng, Biye Li, Weiwei Lü, Peng Cheng, Jianhao Zhang, Xiaoyu Zhang, Liang Zeng, et al. 2024. Skywork-MoE: A Deep Dive into Training Techniques for Mixture-of-Experts Language Models. *arXiv preprint arXiv:2406.06563* (2024).
 - [54] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. Astra-sim2. 0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 283–294.
 - [55] Fuzhao Xue, Zian Zheng, Yao Fu, Jinjie Ni, Zangwei Zheng, Wangchunshu Zhou, and Yang You. 2024. Openmoe: An early effort on open mixture-of-experts language models. *arXiv preprint arXiv:2402.01739* (2024).
 - [56] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. 2022. Glm-130b: An open bilingual pre-trained model. *arXiv preprint arXiv:2210.02414* (2022).
 - [57] Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, et al. 2025. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. *arXiv preprint arXiv:2505.09343* (2025).
 - [58] Liangyu Zhao, Saeed Maleki, Ziyue Yang, Hossein Pourreza, Aashaka Shah, Changho Hwang, and Arvind Krishnamurthy. 2024. ForestColl: Efficient Collective Communications on Heterogeneous Network Fabrics. *arXiv preprint arXiv:2402.06787* (2024).
 - [59] Liangyu Zhao, Siddharth Pal, Tapan Chugh, Weiyang Wang, Jason Fantl, Prithwish Basu, Joud Khoury, and Arvind Krishnamurthy. 2022. Efficient Direct-Connect Topologies for Collective Communications. *arXiv preprint arXiv:2202.03356* (2022).
 - [60] Eckart Zitzler and Lothar Thiele. 1998. Multiobjective optimization using evolutionary algorithms—a comparative case study. In *International conference on parallel problem solving from nature*. Springer, 292–301.

Appendix

Appendices are supporting material that has not been peer-reviewed.

A Use Case 4 of ATOP: Designing a New Data Center for Multi-tenancy

Setup. In this section, we consider the following scenario: establishing a new data center with 16k GPUs, fixedly allocated to four companies, each company using 4k GPUs, respectively ranging from 0–4k, 4k–8k, 8k–12k, and 12k–16k. In this case, we need to consider the topology’s ability to run multiple tasks simultaneously. Therefore, we modify the optimization objectives as follows: For simulating large model training traffic, we consider training one large model per 4k GPUs, resulting in a total of four jobs. The average job completion time across the four jobs for each traffic pattern is used as the optimization objective. For the second-stage end-to-end training evaluation of the topologies, we also use the average iteration time for training four GPT-3-175B models, each utilizing 4k GPUs. Other optimization objectives and configurations remain unchanged, consistent with §4.1 and §4.2.1.

Results. Fig. 14 presents the search results of ATOP in this scenario. We discover that ZCube remains on the Pareto frontier’s reflection point in a multi-tenant scenario, making it the most cost-effective topology.

B Use Case 5 of ATOP: Designing a New Heterogeneous Data Center

Setup. In this section, we consider the following scenario: a data center with 2048 NVIDIA H100 GPUs and 2048 NVIDIA A100 GPUs, where the goal is to build a 4k GPU data center and train models using both types of GPUs simultaneously. The user has already purchased NICs, with each GPU using a single-port NIC. The switch throughput is limited to 12.8 Tbps. These strict topology search space constraints exclude topologies such as BCube, ZCube, and HPN from the search space. The optimization objectives are consistent with those in §4.2.1, but the training workload is now heterogeneous.

Results. The search results are shown in Fig. 15. The best performance topology outperforms ROFT in both performance (+6%) and cost (-11%). The cost-effective topology shows slightly lower performance than ROFT (-2%) but requires only 68% of the cost, demonstrating its cost-effectiveness. In this scenario, ATOP also identifies the Pareto frontier of training time and network cost, where topologies on the frontier represent potential options. Users can select topologies based on their budget. This scenario also demonstrates that ATOP can handle heterogeneous cluster setups and identify better topologies within a strictly constrained search space.

C Definition of Pareto-optimal

In multi-objective optimization problems, there are often complex trade-offs among the objective functions. To ensure fairness among all objectives, the Pareto-optimal [9] solution set is typically used as the solution set for the problem. In the topology multi-objective optimization problem, a Pareto-optimal topology can be formally described as follows:

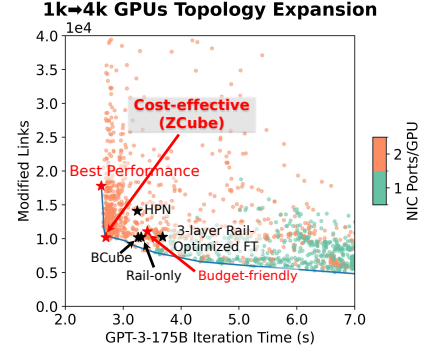


Figure 13: In the search results of Case 3, the comparison between the number of modified links (another cost metric) and training performance.

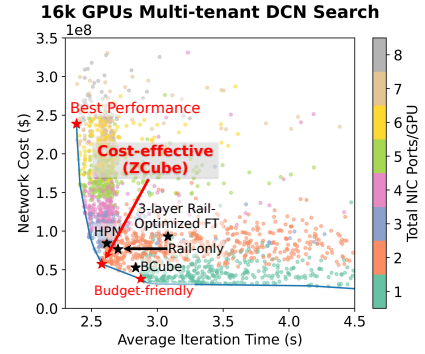


Figure 14: The search results of ATOP when building a new data center for multi-tenancy.

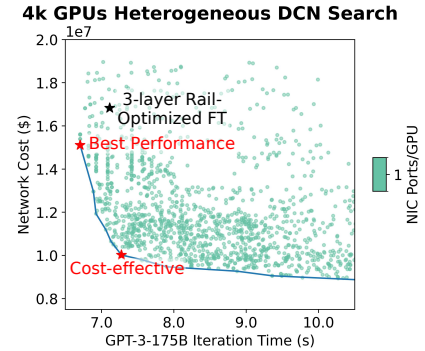


Figure 15: The search results of ATOP when building a new heterogeneous data center with strict search space constraints.

Definition 1 (Pareto-optimal topology). For a multi-objective optimization problem with m objectives:

$$\text{minimize } F(x) = (f_1(x), f_2(x), \dots, f_m(x)),$$

a topology $x \in X$ (where X is the set of all discovered topologies) is Pareto-optimal if and only if:

$$\nexists x' \in X, \text{ such that } \forall i \in [1, m], f_i(x') \leq f_i(x) \text{ and } \exists j \in [1, m], f_j(x') < f_j(x).$$

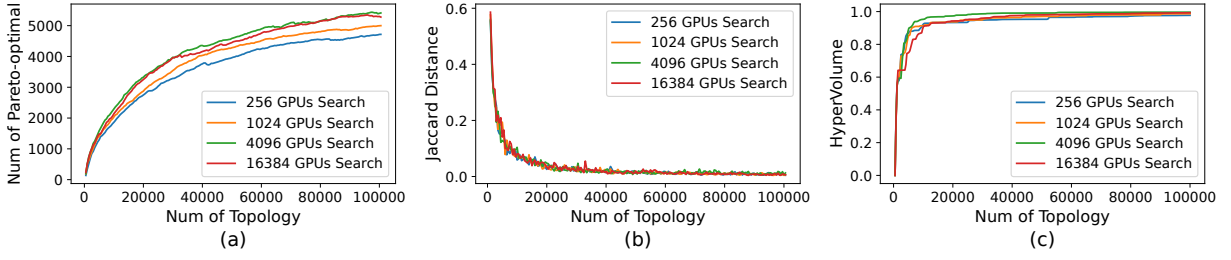


Figure 16: During the ATOP optimization process: (a) The relationship between the number of Pareto-optimal topologies and the total number of topologies generated by ATOP; (b) The Jaccard distance between the Pareto-optimal topology sets of different generations; (c) The relationship between the HyperVolume indicator and the total number of topologies.

The definition of Pareto-optimal indicates that there is no topology $x' \in X$ that performs at least as well as Pareto-optimal topology x across all objectives and strictly better in at least one objective.

Pareto-optimality ensures fairness among optimization objectives, as only topologies that perform poorly across all metrics are excluded from the Pareto-optimal set. This approach also guarantees the feasibility of the 2-stage evaluation since a topology that underperforms across all representative LLM training traffic patterns is likely to exhibit poor end-to-end training performance.

D Convergence and Efficiency of ATOP

In this section, we discuss the convergence and search efficiency of the NSGA-II algorithm in ATOP.

Convergence. We use three metrics to evaluate convergence, as shown in Fig. 16. These metrics are the change in the number of Pareto-optimal topologies as the total number of searched topologies increases, the Jaccard Distance [12] between Pareto-optimal sets across generations in NSGA-II, which measures the variation in the set. In addition, we use HyperVolume [60], a commonly used indicator in multi-objective optimization tasks, to assess convergence. Fig. 16(a) and (b) show the convergence of ATOP on the Pareto-optimal set. Fig. 16(c) shows the convergence of ATOP on the HyperVolume indicator. As the search progresses, the number of Pareto-optimal topologies stabilizes, with no significant changes, and the Jaccard Distance decreases and stabilizes, indicating that the Pareto-optimal set stabilizes in the later stages of the search.

Efficiency. In our configuration, we searched 100,000 topologies for each GPU scale. For 256 GPUs, the total time for ATOP search and 2-stage evaluation is 6.5 hours; for 1k GPUs, it is 10.6 hours; for 4k GPUs, it is 25.4 hours; and for 16k GPUs, it is 71.2 hours. Most of this time is spent on topology evaluation. These times are reasonable compared to the several months typically required to deploy a data center.

E Proof of ZCube Diameter

To prove the diameter of $\text{ZCube}(n, k)$, we first need to establish the following lemma:

Lemma 1. *In $\text{ZCube}(n, k + 1)$, the longest shortest path from any level- k switch to any GPU is at most k hops.*

Proof: This lemma can be proved using mathematical induction. For $\text{ZCube}(n, 1)$, a single switch connects to n GPUs, resulting in

a path length of 0 hops. Assume for $\text{ZCube}(n, k)$ that the longest shortest path from any level- $(k - 1)$ switch to any GPU is at most $k - 1$ hops. When constructing $\text{ZCube}(n, k + 1)$ using n $\text{ZCube}(n, k)$ and n^k switches, each level- $(k - 1)$ switch in a $\text{ZCube}(n, k)$ connects to n different level- k switches, and this process is repeated n times. Thus, any level- k switch is connected to a level- $(k - 1)$ switch in each $\text{ZCube}(n, k)$. By the induction hypothesis, the longest path from a level- k switch to any GPU is $1 + (k - 1) = k$ hops.

Next, we prove the network diameter of $\text{ZCube}(n, k)$.

Theorem 1. *The diameter, defined as the longest shortest path hop count among all the GPU pairs, of a $\text{ZCube}(n, k)$, is k .*

Proof: We can construct a path between any pair of GPUs in $\text{ZCube}(n, k)$ as follows: the source GPU sends data through its k -th port to a level- $(k - 1)$ switch. According to Lemma 1, the longest path from any level- $(k - 1)$ switch to any GPU is at most $k - 1$ hops. By concatenating these two paths, the total path length is k hops. Therefore, the path length between any pair of GPUs does not exceed k hops.

Theorem 2. *The diameter of $\text{ZCube}(n, 3)$ -partial is 4.*

Proof: By the definition of $\text{ZCube}(n, 3)$ -partial, it consists of n $\text{ZCube}(n, 2)$ structures and $n^2/2$ level-2 switches. If the source and destination GPUs are within the same $\text{ZCube}(n, 2)$, their communication path length does not exceed 2 hops (Theorem 1). If they are in different $\text{ZCube}(n, 2)$, the path can be constructed as follows: the source GPU sends data through the second port to a level-1 switch. The level-1 and level-2 switches are interconnected in a CLOS-like manner, allowing a one-hop transition to the level-1 switch in the destination $\text{ZCube}(n, 2)$. Since level-1 switches form a complete bipartite graph with level-0 switches, the path can directly reach the destination's level-0 switch. In this construction, the total path length does not exceed 4 hops.

F Collective Communication Performance of ZCube

In Fig. 3(a) of §2 and Fig. 12 of §6.2, we show that ZCube and ROFT achieve same all-to-all collective communication performance under ideal load balancing. In this section, we discuss their performance under ECMP-based load balancing.

Non-blocking ROFT achieves full bisection bandwidth. However, theoretical bandwidth does not guarantee practical performance.

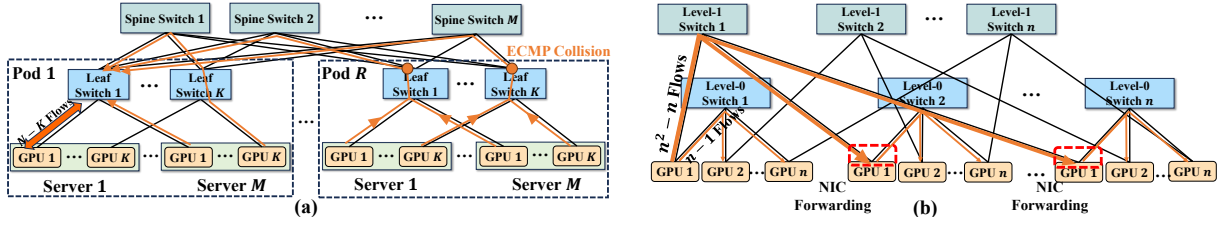


Figure 17: Two scenarios degrade all-to-all performance: (a) ECMP hash collision: In Non-blocking 2-layer Rail-Optimized Fat-Tree (N represents the total number of GPUs, $N = RMK$), ECMP may cause flow collisions during cross-pod all-to-all communication. (b) Not a full bisection bandwidth topology: In BCube($n, 2$), GPU's NIC needs to forward traffic for other GPUs, which makes it not a full bisection bandwidth topology.

ROFT suffers from load balancing inefficiencies under all-to-all traffic due to ECMP hash collisions and hash polarization. As shown in Fig. 17(a), all-to-all flows that traverse different pods may experience hash collisions. The longer path lengths in ROFT increase the likelihood of ECMP collisions. In contrast, ZCube's low-diameter property reduces the number of hops and, consequently, the probability of ECMP collisions.

As for BCube, Fig. 17(b) illustrates that it requires NIC-level packet forwarding during all-to-all communication. This not only consumes NIC bandwidth but also means BCube is not a full bisection bandwidth topology, as each NIC must help forward traffic for others. In ZCube, NIC-level forwarding is unnecessary.

G Fault Tolerance of ZCube

For the fault tolerance of ZCube, we discuss two scenarios: single-switch failures and link failures. ZCube demonstrates excellent performance under both types of failures.

Single-switch Failures. ZCube shows robustness against single-switch failures. When a single switch fails, the GPU connected to the failed switch can switch to another NIC port.

We present the performance degradation of the 4096 GPUs ZCube when training the GPT-3 175B model under a single switch failure in Fig. 3(b) based on our flow-level simulation. The degradation of ZCube is only 2.8%, while ROFT experiences a 46.9% degradation and HPN experiences a 9.0% degradation, while reducing 32% of the cost compared to HPN.

We also use the APL_{fail} introduced in §3.4 to evaluate the fault tolerance. We compute the average path length between GPU pairs without switch failures, denoted as APL . To assess the change before and after failure, we use the Average Path Stretch (APS), defined as $APS = (APL_{\text{fail}} - APL)/APL$. We choose APL_{fail} over APS as the optimization objective in topology searching because APL_{fail} accounts for both performance under single-switch failure and the topology's diameter. We consider both scenarios where 8-GPU servers are interconnected and where no inter-server connections exist. For the scenario with inter-server connections, we treat the communication path length between GPU pairs within the same server as 1, where the 8 GPUs form a full-mesh topology. The results, shown in Tab. 3, indicate that ZCube($n, 2$) achieves the lowest APL and APL_{fail} due to its low network diameter and is less affected by single-switch failures compared to ROFT and Dragonfly topologies. The HPN topology shows no change in average path length between GPU pairs before and after failure due to its dual-ToR architecture.

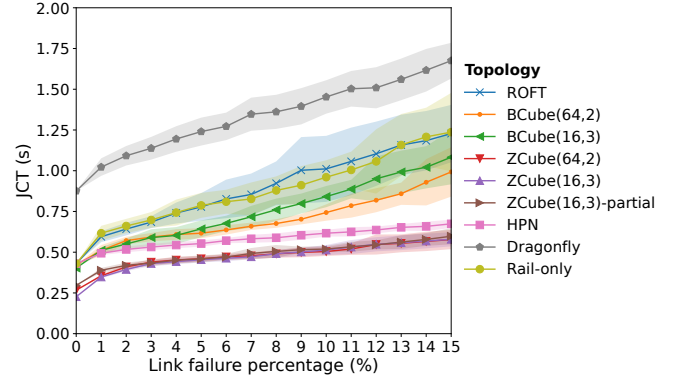


Figure 18: The average JCT for group all-to-all communication under different topologies with link failures on 4096 GPUs, with shading representing the standard deviation of the JCT.

Link Failures. ZCube demonstrates graceful performance degradation under link failures. We randomly select a portion of the network links to fail and measure the job completion time (JCT) of a 10 GB group all-to-all communication on the faulty topology using our flow-level network simulator. We evaluate 8 topologies at the 4096 GPU scale, varying the failure rate from 1% to 15% of RDMA network links, and repeat each test 100 times to compute the mean and standard deviation of the JCT. Intra-server connections within 8-GPU servers are considered but do not fail, consistent with the settings in §4.1. The results, shown in Fig. 18, indicate that ZCube exhibits a more graceful performance degradation under link failures and with smaller standard deviations.

H Comparison between Our Packet-level Network Simulator and Real-world Testbed

In this section, we compare our packet-level network simulator with the real-world testbed described in §6.2. We replicate the same topology configuration from §6.2 in the simulator and evaluate two collective communication primitives: all-reduce and all-to-all. The simulator setup follows the configuration in §6.1, using packet spraying for routing and load balancing. As shown in Fig. 19, the average error between the simulator and the real-world testbed is only 5%.

Table 3: The average communication path length between GPU pairs and average path stretch for different topologies, both before and after a single-switch failure. * indicates that considering intra-server interconnection, the communication path length within a server is considered to be 1.

GPU Scale	Topology	APL	APL _{fail}	APS($\times 10^{-4}$)	APL*	APL _{fail} *	APS*($\times 10^{-4}$)
1024 GPUs	ROFT	5.47214	-	-	5.34897	5.36011	20.82644
	Rail-only	-	-	-	4.61388	4.61631	5.26672
	Dragonfly	4.67867	-	-	4.62732	4.63901	25.26300
	BCube(32,2)	3.87879	3.88258	9.77109	3.65982	3.66224	6.61235
	HPN	3.93939	3.93939	0	3.76918	3.76918	0
	ZCube(32,2)	2.93939	2.94129	6.46393	2.93255	2.93423	5.72880
4096 GPUs	ROFT	5.48523	-	-	5.42711	5.43301	10.87135
	Rail-only	-	-	-	4.74725	4.74851	2.65417
	Dragonfly	4.79810	-	-	4.77709	4.78088	7.93370
	BCube(64,2)	3.93846	3.93942	2.43750	3.82906	3.82973	1.74978
	BCube(16,3)	5.62637	5.62643	0.10664	5.18877	5.18889	0.23127
	HPN	3.93797	3.93797	0	3.71575	3.71575	0
	ZCube(64,2)	2.96923	2.96971	1.61658	2.96752	2.96798	1.55012
	ZCube(16,3)	3.80975	3.81007	0.83995	3.78561	3.78591	0.79247
16384 GPUs	ZCube(16,3)-partial	4.75092	4.75130	0.79985	4.74921	4.74959	0.80013
	ROFT	5.49240	-	-	5.46420	5.46724	5.56349
	Rail-only	-	-	-	4.81182	4.81246	1.33006
	Dragonfly	4.85422	-	-	4.85379	4.85455	1.56579
	BCube(128,2)	3.96899	3.96923	0.60469	3.91430	3.91448	0.45985
	HPN	3.98450	3.98450	0	3.92895	3.92895	0
	ZCube(128,2)	2.98450	2.98462	0.40208	2.98407	2.98419	0.40214

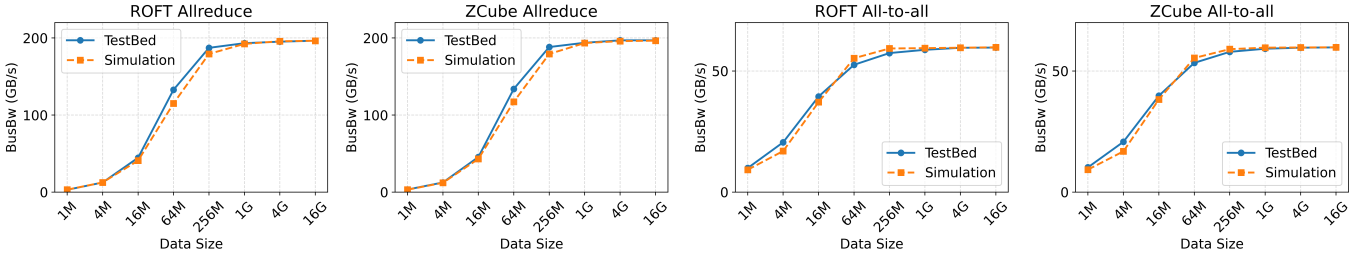


Figure 19: Comparison between packet-level network simulation (with packet spraying for load balancing) and the real-world testbed in §6.2.

I Comparison between Our Flow-level Network Simulator and NS-3

In this section, we compare the relative error between the NS-3 network simulator’s results and our flow-level network simulator’s results. Due to NS-3’s lower simulation efficiency, we only compare topologies at the 64 and 256 GPU scales, including ROFT, Dragonfly, BCube, HPN, and ZCube. The configuration for the flow-level simulator is consistent with §4.1. For the configuration in NS-3, we set the link latency to 5 μ s, use ECMP for load-balancing.

We first evaluate the relative error in training iteration time for simulating the training of a GPT-3-22B model using “Astra-Sim 2.0 + NS-3” and “Astra-Sim 2.0 + flow-level simulator” at 64 and 256 GPU scales, as shown in Tab. 4. The average relative error in training iteration time is only 1.5%, demonstrating that the flow-level simulation accuracy is sufficient for evaluating the performance of topologies. Additionally, we evaluate the relative error in job completion time

for a 64 MB all-to-all collective communication task at 64 GPUs in a scenario involving flow conflicts and congestion. As shown in Tab. 5, the average relative error in job completion time is only 4.3%, further validating the precision of the flow-level simulator. Finally, we present the cumulative distribution function (CDF) comparison of flow completion times (FCT) for the GPT-3-22B model training workload on a 256 GPUs ZCube topology and the CDF comparison of FCT for all-to-all collective communication on a 64 GPUs ZCube topology, as shown in Fig. 20. The results further validate the high fidelity of the flow-level network simulator.

J Evaluation of Different Hyperparameter Optimization Algorithms

Currently, the ATOP topology optimizer supports several hyperparameter optimization algorithms, including NSGA-II, Bayesian optimization [5], Quasi Monte Carlo (QMC) [43], and Random Search.

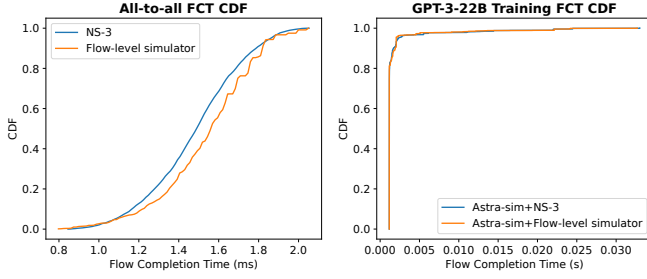


Figure 20: Comparison of the CDF of flow completion times between NS-3 and flow-level simulators.

Table 4: Simulation results of GPT-3-22B model training time, using different network simulators as the network backend of Astra-Sim 2.0 [54].

GPU Scale	Topology	GPT-3-22B Iter Time (s)		Relative Error
		NS-3 Results	Flow-level Results	
64 GPUs	ROFT	2.497	2.457	1.6%
	Dragonfly	3.383	3.257	3.7%
	BCube(8,2)	2.479	2.423	2.3%
	HPN	2.413	2.395	0.8%
	ZCube(8,2)	2.373	2.364	0.3%
256 GPUs	ROFT	2.421	2.455	1.4%
	Dragonfly	3.121	3.220	3.2%
	BCube(16,2)	2.277	2.295	0.8%
	HPN	2.247	2.235	0.5%
	ZCube(16,2)	2.198	2.193	0.2%

Table 5: 64 MB All-to-all collective communication job completion time, using different network simulators.

Topology	All-to-All (ms)		Relative Error
	NS-3 Results	Flow-level Results	
ROFT	4.511	4.841	7.3%
Dragonfly	5.204	5.473	5.2%
BCube(8,2)	3.977	4.057	2.0%
HPN	2.608	2.742	5.1%
ZCube(8,2)	2.052	2.011	2.0%

Through practical experimentation, we discover that NSGA-II performs best in our defined problem and search space, offering a balance of efficiency and performance. Using NSGA-II, we completed a search of 100000 topologies for 1024 GPUs within 10.6 hours on our TestBed. In the same environment and timeframe, Bayesian optimization explored only 1000 topologies, QMC explored 26000 topologies, and random search explored 130000 topologies. None of these algorithms lead to the convergence of the Pareto-optimal topology set, nor do they identify the Pareto frontier between GPT-3 training time and network cost.

K Diagrams and Comparison between ROFT, Rail-only, HPN, and ZCube on Building 16384 GPU Cluster

In this section, we present the topology diagrams of ROFT (Fig. 21), Rail-only (Fig. 22), HPN (Fig. 23) and ZCube(128,2) (Fig. 24) for building a 16384 GPU cluster using the commonly adopted Broadcom Tomahawk 5 [22] switch (with a switch chip throughput of 51.2 Tbps). All servers are configured with 8 GPUs and $8 \times 400\text{G}$ NICs, which is one of the most common setups [39]. All ports of all

switches are fully utilized to minimize the number of switches as much as possible.

We also summarize in Tab. 6 the number of switches and cables required for building these topologies, demonstrating the low cost of ZCube. These topologies provide the same ForestColl all-gather performance. However, due to ZCube’s low network diameter, it accelerates PP traffic, resulting in superior training performance compared to other topologies, as discussed in §6.1.

Table 6: Switch and cable counts required by different topologies when building a 16384 GPU cluster with 51.2 Tbps switches.

Topology	Switch (51.2 Tbps)	Cables (Num $\times$ Bandwidth)
ROFT	640	49152 $\times$ 400G
Rail-only	384	32768 $\times$ 400G
HPN	384	16384 $\times$ 400G+32768 $\times$ 200G
ZCube(128,2)	256	49152$\times$200G

L Configurations of the Evaluated Topology

In Tab. 7, we present the configurations of the topologies evaluated in §6.1, including the number of switches and radix, the number of cables (both copper and fiber optics), as well as the number of optical modules and network interface card (NIC) ports. For the ROFT, we construct a non-blocking 3-layer Rail-Optimized Fat-Tree. For the Dragonfly topology, we set its parameters as $a = 2p = 2h$ [32]. For the HPN topology, we align with Alibaba’s HPN [44] design. The connection within each segment follows a multi-rail strategy with a dual ToR architecture. We also use a non-blocking configuration. Specifically, for the 16k GPU scale, each ToR switch has a downlink bandwidth of $128 \times 200\text{ Gbps}$ and an uplink bandwidth of $64 \times 400\text{ Gbps}$.

For ROFT and HPN, we use fiber optics, as they employ a multi-rail interconnection method between the NICs and ToR switches, which requires longer distances. For BCube and ZCube, we use copper cables between the NICs and the first-layer switches, as the NICs on the same server connect to the same ToR. Fiber optics are used for other interconnections. For Dragonfly, copper cables are used for interconnections within a group, while fiber optics are used for interconnections between groups.

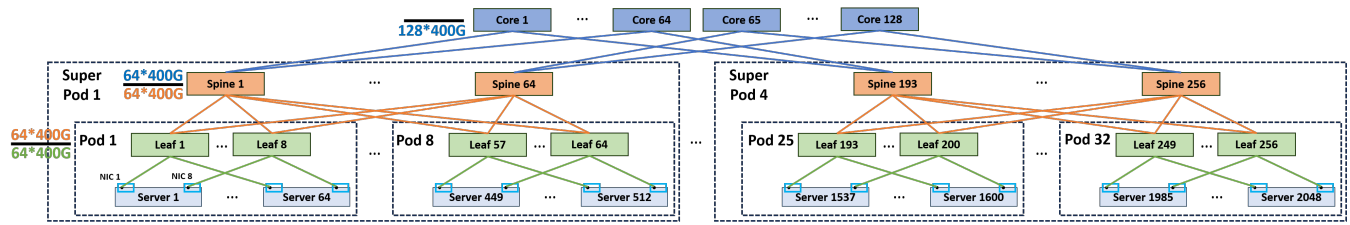


Figure 21: ROFT topology for a 16384 GPU cluster based on 51.2 Tbps switches.

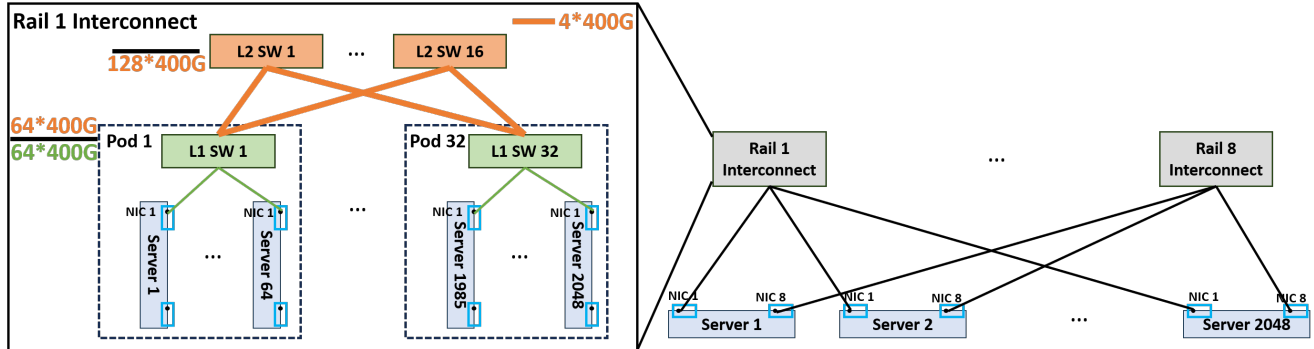


Figure 22: Rail-only topology for a 16384 GPU cluster based on 51.2 Tbps switches. Each Rail-interconnection adopts a 2-layer CLOS architecture, consistent with [51] and [57].

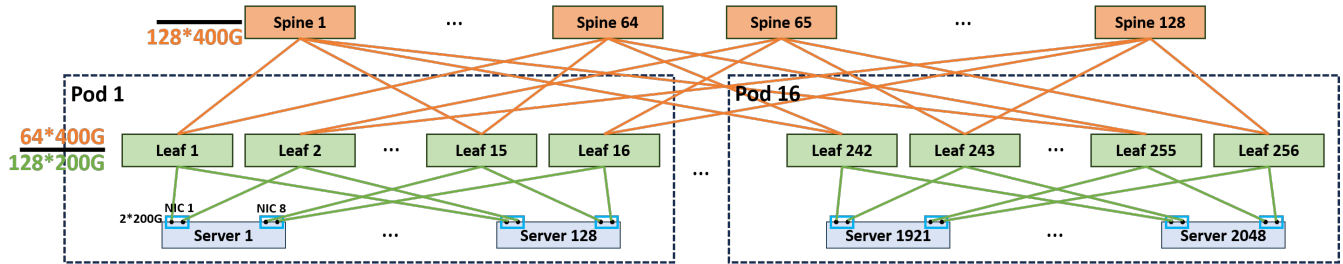


Figure 23: HPN topology (dual-port designs for ROFT) for a 16384 GPU cluster based on 51.2 Tbps switches.

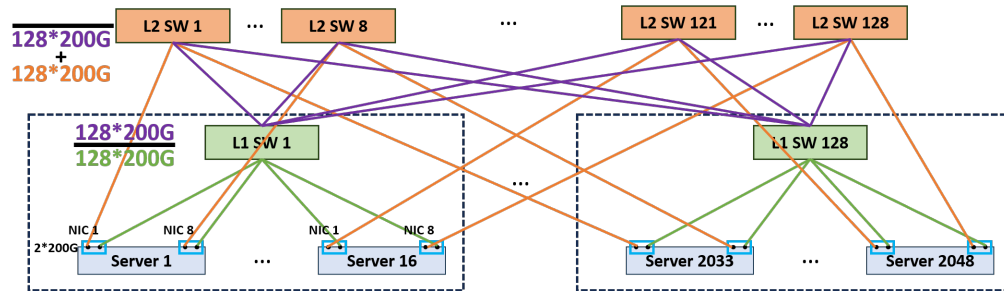


Figure 24: ZCube(128,2) topology for a 16384 GPU cluster based on 51.2 Tbps switches.

Table 7: Configurations of the evaluated topology in §6.

GPU Scale	Topology	Switches	Radix	Cables		Optical Module	NIC Port/GPU
				Copper	Fiber		
1024 GPUs	ROFT	160	32×400G	0	3072	6144	1
	Rail-only	96	32×400G	0	2048	4096	1
	Dragonfly	264	15×400G	1980	528	1056	1
	BCube(32,2)	64	32×200G	1024	1024	2048	2
	HPN	96	64×200G	0	3072	6144	2
	ZCube(32,2)	64	64×200G	1024	2048	4096	2
4096 GPUs	ROFT	320	64×400G	0	12288	24576	1
	Rail-only	192	64×400G	0	8192	16384	1
	Dragonfly	876	23×400G	10074	2628	5256	1
	BCube(64,2)	128	64×200G	4096	4096	8192	2
	HPN	192	128×200G	0	12288	24576	2
	ZCube(16,3)-partial	640	48×200G	4096	12288	24576	2
16384 GPUs	ZCube(64,2)	128	128×200G	4096	8192	16384	2
	ROFT	640	128×400G	0	49152	98304	1
	Rail-only	384	128×400G	0	32768	65536	1
	Dragonfly	2064	31×400G	31992	8256	16512	1
	BCube(128,2)	256	128×200G	16384	16384	32768	2
	HPN (16 Segment)	384	256×200G	0	49152	98304	2
	ZCube(128,2)	256	256×200G	16384	32768	65536	2

Table 8: Detailed model hyperparameters and training configurations in §4.1.

GPU Scale	Model Size	Attention Heads	Hidden Size	Layers	TP Size	DP Size	PP Size	Global Batch Size	Micro-batch Size	Number of Interleaved Stages
256 GPUs	22B	64	6144	48	8	4	8	384	1	3
1024 GPUs	175B	96	12288	96	8	16	8	1536	1	3
4096 GPUs	175B	96	12288	96	8	64	8	1536	1	3
16384 GPUs	175B	96	12288	96	8	256	8	6144	1	3